

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



Nguyễn Hà An

Xây dựng chatbot cho website bán hàng điện tử

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CLC
Ngành: Khoa học Máy tính

HÀ NỘI - 2025

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Nguyễn Hà An

Xây dựng chatbot cho website bán hàng điện tử

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CLC

Ngành: Khoa học Máy tính

Cán bộ hướng dẫn: PGS.TS. Nguyễn Phương Thái

(Ký tên)

HÀ NỘI - 2025

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn đến Khoa Công nghệ thông tin, Trường Đại học Công nghệ, Đại học Quốc Gia Hà Nội đã hỗ trợ, tạo điều kiện tốt nhất cho em để hoàn thành khóa luận tốt nghiệp này.

Em cũng xin được bày tỏ lòng biết ơn sâu sắc tới thầy PGS.TS. Nguyễn Phương Thái đã dành thời gian quý báu, tận tình hướng dẫn, đóng góp những ý kiến xác đáng để em có thể hoàn thành khóa luận một cách tốt nhất.

Cuối cùng, em xin gửi lời cảm ơn sâu sắc đến các thầy cô giảng viên Trường Đại học Công nghệ, Đại học Quốc Gia Hà Nội đã luôn luôn chỉ dạy và giúp đỡ em, đặt cho em nền tảng vững chắc để có thể phát triển trong ngành Công nghệ Thông tin. Em cũng xin cảm ơn bạn bè và gia đình đã chăm sóc, giúp đỡ, làm chỗ dựa tinh thần to lớn cho em vượt qua những khó khăn trong quá trình học tập.

Do thời gian có hạn, kinh nghiệm còn hạn chế, khóa luận tốt nghiệp không tránh khỏi những thiếu sót. Em rất mong nhận được sự đóng góp của các thầy cô giảng viên để khóa luận được hoàn thiện hơn.

Em xin chân thành cảm ơn!

LỜI CAM ĐOAN

Em xin cam đoan nội dung được trình bày trong khóa luận tốt nghiệp " Xây dựng chatbot cho website bán hàng điện tử " đều do em thực hiện dưới sự hướng dẫn của PGS.TS. Nguyễn Phương Thái. Em không sao chép các tài liệu, công trình nghiên cứu của người khác mà không chỉ rõ trong tài liệu tham khảo. Các tài liệu tham khảo trong quá trình thực hiện đều được em chú thích cụ thể và rõ ràng trong danh mục tài liệu tham khảo trong khóa luận. Nếu phát hiện bất kỳ sự gian lận nào, em xin chịu trách nhiệm trước Hội đồng, cũng như kết quả khóa luận của mình.

Hà Nội, ngày 25 tháng 11 năm 2025

Sinh viên

Nguyễn Hà An

Tóm tắt

Thương mại điện tử đang phát triển mạnh mẽ tại Việt Nam với tốc độ 20-25%/năm, đặc biệt trong lĩnh vực điện thoại thông minh và máy tính bảng. Tuy nhiên, các nền tảng e-commerce gặp khó khăn trong việc cung cấp dịch vụ chăm sóc khách hàng 24/7, dẫn đến trải nghiệm người dùng kém.

Đề tài "Hệ thống Thương mại Điện tử Wireless World với Trợ lý Ảo Thông minh" xây dựng giải pháp toàn diện với hai phần chính: nền tảng thương mại điện tử (quản lý sản phẩm, giỏ hàng, đặt hàng) và trợ lý ảo AI sử dụng công nghệ RAG (Retrieval-Augmented Generation).

Chatbot AI được xây dựng theo kiến trúc RAG 5 bước: cache → intent detection → retrieval → augmentation → generation. Chatbot trả lời câu hỏi về giá, tồn kho, thông số kỹ thuật dựa trên dữ liệu real-time. Hệ thống áp dụng smart filtering (giảm 90% token), caching (85% hit rate), rate limiting để tối ưu chi phí.

Công nghệ: PHP 8.2, MySQL 8.0, Bootstrap 5, JavaScript ES6+, Google Gemini API (2.5 Flash). Kiến trúc 3-tier với bảo mật: prepared statements, XSS prevention, session management.

Từ khóa: Thương mại điện tử, RAG, Chatbot AI, PHP, MySQL, Google Gemini API, E-commerce.

Mục Lục

LỜI CẢM ƠN.....	3
LỜI CAM ĐOAN	4
Tóm tắt.....	5
Mục Lục.....	6
Danh sách hình vẽ.....	8
Danh sách biểu đồ.....	8
BẢNG CÁC KÝ HIỆU, CHỮ VIẾT TẮT	9
CHƯƠNG 1: GIỚI THIỆU	11
1.1. Bối cảnh và lý do chọn đề tài.....	11
1.2. Phát biểu vấn đề.....	12
1.3. Mục tiêu nghiên cứu	13
1.4. Phạm vi nghiên cứu	14
1.5. Phương pháp nghiên cứu.....	15
1.6. Ý nghĩa của đề tài.....	16
1.7. Cấu trúc khóa luận	17
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ	20
2.1. Tổng quan về thương mại điện tử.....	20
2.2. Công nghệ RAG (Retrieval-Augmented Generation)	21
2.3. Các công nghệ triển khai	23
2.3.1. PHP và quản lý cơ sở dữ liệu.....	23
2.3.2. Framework và công nghệ frontend	24
2.3.3. Google Gemini API	25
2.4. Khảo sát các hệ thống hiện có	27
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	29
3.1. Phân tích yêu cầu hệ thống.....	29
3.2. Thiết kế kiến trúc hệ thống.....	31
3.3. Thiết kế kiến trúc chatbot RAG.....	34
3.4. Thiết kế cơ sở dữ liệu.....	38
3.4.1. Sơ đồ ERD (Entity Relationship Diagram).....	38
3.5. Thiết kế giao diện người dung.....	38

3.5.1. Nguyên tắc thiết kế UI/UX.....	38
3.5.2. Wireframe các trang chính.....	39
3.6. Thiết kế API và bảo mật.....	44
CHƯƠNG 4: TRIỂN KHAI HỆ THỐNG	47
4.1. Môi trường triển khai.....	47
4.2. Triển khai cơ sở dữ liệu.....	48
4.3. Triển khai module Authentication.....	50
4.4. Triển khai AI Chatbot với RAG	51
4.5. Tối ưu hóa hiệu suất.....	54
CHƯƠNG 5: KIỂM THỬ HỆ THỐNG	57
5.1. Phương pháp kiểm thử.....	57
5.2. Unit Testing.....	57
5.2.1. Kiểm thử Module Xác thực và Quản lý Người dùng.....	57
5.2.2. Kiểm thử Module Quản lý Sản phẩm	59
5.2.3. Kiểm thử Module Upload Hình ảnh.....	59
5.3. Integration Testing.....	61
5.3.1. Luồng Mua hàng End-to-End	61
5.3.2. Luồng RAG Chatbot Chi tiết	61
5.4. Performance Testing.....	64
5.4.1. Kiểm thử Load với JMeter	64
5.4.2. Database Performance Analysis.....	64
5.4.3. Kiểm thử Memory Leak	66
5.5. Security Testing	67
5.5.1. SQL Injection Testing.....	67
5.5.2. Cross-Site Scripting (XSS) Testing.....	68
5.5.3. Session Hijacking Testing.....	69
5.5.4. CSRF (Cross-Site Request Forgery) Testing	69
5.5.5. Brute Force Testing.....	70
5.5.6. File Upload Security Testing	70
5.5.7. Input Validation Testing.....	71
Chương 6: Kết Luận Và Hướng Phát Triển.....	73
6.1. Kết Luận.....	73
6.2. Hạn Chế.....	73

6.3. Hướng Phát Triển.....	74
6.4. Kinh Nghiệm Rút Ra	74
TÀI LIỆU THAM KHẢO.....	76

Danh sách hình vẽ

Hình 3.1. Giao diện trang chủ 1.....	41
Hình 3.2. Giao diện trang chủ 2.....	41
Hình 3.3. Giao diện trang chủ 3.....	41
Hình 3.4. Giao diện trang chủ 4.....	42
Hình 3.5. Giao diện chi tiết sản phẩm.....	43
Hình 3.6. Giao diện chatbot.....	43
Hình 3.7. Giao diện giỏ hàng.....	44

Danh sách biểu đồ

Bảng 3.1. Yêu cầu phi chức năng	
Bảng 3.2. mô hình 3-Tier Architecture	
Bảng 3.3. Kiến trúc AI Chatbot với RAG	
Bảng 3.4. Sơ đồ ERD	
Bảng 4.1. Danh sách phần mềm và công cụ sử dụng dự án	
Bảng 4.2. Bảng chỉ số đo lường hiệu suất)	
Bảng 5.1. Bảng kiểm thử tích hợp kiểm tra sự tương tác giữa các modules khác nhau	

BẢNG CÁC KÝ HIỆU, CHỮ VIẾT TẮT

Ký hiệu	Tên đầy đủ	Giải thích
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
AR	Augmented Reality	Thực tế tăng cường
CPU	Central Processing Unit	Bộ xử lý trung tâm
CSRF	Cross-Site Request Forgery	Tấn công giả mạo yêu cầu giữa các trang
CSS	Cascading Style Sheets	Bảng định kiểu theo tầng
GB	Gigabyte	Gigabyte (đơn vị đo dung lượng)
HTML	HyperText Markup Language	Ngôn ngữ đánh dấu siêu văn bản
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
HTTPS	HyperText Transfer Protocol Secure	Giao thức truyền tải siêu văn bản bảo mật
IDE	Integrated Development Environment	Môi trường phát triển tích hợp
iOS	iPhone Operating System	Hệ điều hành iPhone
JSON	JavaScript Object Notation	Định dạng dữ liệu JavaScript
MB	Megabyte	Megabyte (đơn vị đo dung lượng)
MySQL	My Structured Query Language	Hệ quản trị cơ sở dữ liệu quan hệ
PDO	PHP Data Objects	Đối tượng dữ liệu PHP
PHP	PHP: Hypertext Preprocessor	Ngôn ngữ lập trình kịch bản
RAG	Retrieval-Augmented Generation	Sinh văn bản tăng cường truy xuất
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
SEO	Search Engine Optimization	Tối ưu hóa công cụ tìm kiếm
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
SSD	Solid State Drive	Ổ cứng thể rắn
TTL	Time To Live	Thời gian tồn tại (của dữ liệu cache)
UAT	User Acceptance Testing	Kiểm thử chấp nhận người dùng
UI/UX	User Interface/User Experience	Giao diện người dùng/Trải nghiệm người dùng
URL	Uniform Resource Locator	Định vị tài nguyên thống nhất

UTF-8	Unicode Transformation Format - 8-bit	Định dạng chuyển đổi Unicode 8 bit
VR	Virtual Reality	Thực tế ảo
WCAG	Web Content Accessibility Guidelines	Hướng dẫn khả năng tiếp cận nội dung web
XAMPP	Cross-Platform Apache MySQL PHP Perl	Gói phần mềm máy chủ đa nền tảng
XSS	Cross-Site Scripting	Tấn công chèn mã độc giữa các trang

CHƯƠNG 1: GIỚI THIỆU

1.1. Bối cảnh và lý do chọn đề tài

Trong bối cảnh cuộc cách mạng công nghiệp 4.0 đang diễn ra mạnh mẽ, thương mại điện tử đã trở thành một xu hướng tất yếu và đóng vai trò then chốt trong việc chuyển đổi số của các doanh nghiệp. Theo báo cáo của Hiệp hội Thương mại Điện tử Việt Nam, thị trường thương mại điện tử tại Việt Nam đang có tốc độ tăng trưởng ấn tượng với mức trung bình 20-25% mỗi năm. Đặc biệt, lĩnh vực kinh doanh thiết bị di động, bao gồm điện thoại thông minh và máy tính bảng, đang chiếm một phần đáng kể trong tổng doanh thu thương mại điện tử do nhu cầu ngày càng cao của người tiêu dùng về các thiết bị công nghệ hiện đại.

Tuy nhiên, bên cạnh những cơ hội to lớn, các nền tảng thương mại điện tử hiện nay đang phải đối mặt với nhiều thách thức quan trọng. Một trong những khó khăn lớn nhất là việc cung cấp dịch vụ chăm sóc khách hàng liên tục 24/7 để đáp ứng nhu cầu tư vấn và hỗ trợ của người mua. Các phương pháp truyền thống như sử dụng đường dây nóng, email hay live chat với nhân viên đều đòi hỏi chi phí nhân sự rất cao và không thể đảm bảo phục vụ khách hàng vào bất kỳ thời điểm nào trong ngày. Điều này dẫn đến tình trạng khách hàng phải chờ đợi lâu để được giải đáp thắc mắc, từ đó ảnh hưởng tiêu cực đến trải nghiệm mua sắm và khả năng chuyển đổi từ người xem thành người mua.

Với sự phát triển vượt bậc của trí tuệ nhân tạo trong những năm gần đây, các công nghệ như xử lý ngôn ngữ tự nhiên và học máy đã mở ra nhiều cơ hội mới trong việc tự động hóa dịch vụ khách hàng. Đặc biệt, kiến trúc RAG (Retrieval-Augmented Generation) đã chứng minh hiệu quả trong việc kết hợp khả năng sinh văn bản của các mô hình ngôn ngữ lớn với dữ liệu thực tế từ hệ thống, giúp tạo ra những câu trả lời chính xác và có căn cứ. Tuy nhiên, việc triển khai các giải pháp AI này vào thực tế cần phải giải quyết nhiều vấn đề về chi phí, hiệu suất và bảo mật.

Xuất phát từ những nhu cầu thực tiễn và tiềm năng công nghệ nêu trên, đề tài "Hệ thống Thương mại Điện tử Wireless World với Trợ lý Ảo Thông minh" được lựa chọn nhằm xây dựng một giải pháp toàn diện, kết hợp nền tảng thương mại điện tử với trợ lý ảo AI sử dụng công nghệ RAG để cải thiện trải nghiệm khách hàng đồng thời tối ưu hóa chi phí vận hành.

1.2. Phát biểu vấn đề

Qua quá trình khảo sát và nghiên cứu các hệ thống thương mại điện tử hiện có trên thị trường, tác giả đã xác định được một số vấn đề quan trọng cần được giải quyết.

Thứ nhất, vấn đề về khả năng hỗ trợ khách hàng tự động vẫn chưa được đáp ứng đầy đủ. Trong thực tế, phần lớn khách hàng truy cập vào các website thương mại điện tử đều có những câu hỏi cơ bản về sản phẩm như giá cả, tình trạng tồn kho, thông số kỹ thuật, hoặc các chính sách bảo hành và đổi trả. Tuy nhiên, họ thường không thể nhận được câu trả lời ngay lập tức, đặc biệt là vào các thời điểm ngoài giờ làm việc hoặc vào cuối tuần và ngày lễ. Điều này không chỉ làm giảm sự hài lòng của khách hàng mà còn dẫn đến việc mất đi cơ hội bán hàng. Đối với các doanh nghiệp vừa và nhỏ, việc duy trì một đội ngũ nhân viên chăm sóc khách hàng làm việc 24/7 là gần như không khả thi do chi phí quá cao.

Thứ hai, các chatbot truyền thống thường gặp phải vấn đề về độ chính xác và tính cập nhật của thông tin. Hầu hết các chatbot hiện tại được lập trình dựa trên các kịch bản cố định hoặc sử dụng dữ liệu tĩnh đã được huấn luyện trước. Điều này dẫn đến tình trạng chatbot không thể cung cấp thông tin chính xác về giá cả hoặc tình trạng hàng tồn kho theo thời gian thực, vì những thông tin này thường xuyên thay đổi trong hệ thống. Khi khách hàng nhận được thông tin không chính xác, họ sẽ mất lòng tin vào hệ thống và có thể rời bỏ trang web.

Thứ ba, chi phí triển khai và vận hành các giải pháp AI là một rào cản lớn. Việc sử dụng các API của các mô hình ngôn ngữ lớn như GPT hay Gemini đòi hỏi chi phí dựa trên số lượng token được xử lý. Nếu không được tối ưu hóa cẩn thận, chi phí này có thể tăng lên rất nhanh, đặc biệt khi hệ thống có lượng người dùng lớn. Ngoài ra, việc truy vấn cơ sở dữ liệu liên tục để lấy thông tin sản phẩm cũng gây áp lực lên hiệu suất của hệ thống, có thể dẫn đến tình trạng chậm trễ hoặc quá tải.

Thứ tư, các vấn đề về bảo mật và spam là mối quan ngại thường trực đối với các hệ thống trực tuyến. Hệ thống thương mại điện tử dễ trở thành mục tiêu của các cuộc tấn công như SQL Injection, Cross-Site Scripting (XSS), hoặc các hình thức spam chatbot nhằm tiêu tốn tài nguyên hệ thống. Việc thiếu các cơ chế bảo vệ thích hợp không chỉ gây ra rủi ro về dữ liệu mà còn ảnh hưởng đến uy tín và khả năng vận hành của doanh nghiệp.

Từ những vấn đề được phân tích ở trên, đề tài này tập trung vào việc xây dựng một hệ thống thương mại điện tử toàn diện với trợ lý ảo AI thông minh, nhằm giải quyết đồng thời các thách thức về chất lượng dịch vụ khách hàng, độ chính xác thông tin, tối ưu hóa chi phí và đảm bảo an toàn bảo mật.

1.3. Mục tiêu nghiên cứu

Dựa trên các vấn đề đã được xác định, đề tài đặt ra bốn mục tiêu chính cần đạt được trong quá trình nghiên cứu và triển khai.

Mục tiêu thứ nhất là xây dựng một hệ thống thương mại điện tử hoàn chỉnh và chuyên nghiệp cho lĩnh vực kinh doanh điện thoại thông minh và máy tính bảng. Hệ thống cần cung cấp đầy đủ các chức năng cơ bản và nâng cao của một nền tảng thương mại điện tử hiện đại, bao gồm khả năng quản lý danh mục sản phẩm với nhiều hình ảnh, hệ thống phân loại theo thương hiệu và danh mục, chức năng giỏ hàng với khả năng cập nhật số lượng và xóa sản phẩm, quy trình đặt hàng đơn giản và rõ ràng, cũng như trang quản trị viên để theo dõi và quản lý toàn bộ hoạt động của hệ thống. Giao diện người dùng cần được thiết kế thân thiện, trực quan và đáp ứng tốt trên nhiều thiết bị khác nhau từ máy tính để bàn đến điện thoại di động.

Mục tiêu thứ hai là tích hợp trợ lý ảo AI thông minh sử dụng công nghệ RAG để nâng cao chất lượng dịch vụ khách hàng. Trợ lý ảo này cần có khả năng trả lời tự động các câu hỏi của khách hàng về sản phẩm trong bất kỳ thời điểm nào, không giới hạn bởi giờ làm việc. Đặc biệt, chatbot phải có khả năng cung cấp thông tin chính xác và cập nhật về giá cả, tình trạng tồn kho, cũng như các thông số kỹ thuật của sản phẩm dựa trên dữ liệu thời gian thực từ cơ sở dữ liệu. Ngoài ra, chatbot cần có khả năng hiểu được ý định của người dùng, phân biệt được các loại câu hỏi khác nhau, và đưa ra các gợi ý sản phẩm phù hợp với nhu cầu cụ thể của từng khách hàng. Chatbot cũng cần có khả năng xử lý các cuộc hội thoại tự nhiên, không chỉ giới hạn ở việc trả lời các câu hỏi về sản phẩm mà còn có thể tương tác thân thiện với khách hàng qua các câu chào hỏi và cảm ơn thông thường.

Mục tiêu thứ ba tập trung vào việc tối ưu hóa hiệu suất và giảm thiểu chi phí vận hành thông qua các kỹ thuật kỹ thuật. Hệ thống cần triển khai một cơ chế cache thông minh để lưu trữ tạm thời kết quả của các truy vấn phổ biến, từ đó giảm số lượng truy vấn trực tiếp đến cơ sở dữ liệu và cải thiện thời gian phản hồi. Để giảm chi phí sử dụng API AI, hệ thống cần áp dụng kỹ thuật lọc thông minh (smart filtering) để chỉ gửi những thông tin sản phẩm thực sự cần thiết cho mô hình AI, thay vì gửi toàn bộ dữ liệu. Mục tiêu đặt ra là giảm ít nhất 60-70% lượng token được sử dụng so với phương pháp truyền thống. Hơn nữa, hệ thống cần có các cơ chế kiểm

soát như rate limiting để giới hạn số lượng yêu cầu từ mỗi người dùng trong một khoảng thời gian nhất định, và quota để đảm bảo không vượt quá ngân sách dự kiến cho việc sử dụng API.

Mục tiêu thứ tư là đảm bảo an toàn và bảo mật cho toàn bộ hệ thống. Hệ thống cần được xây dựng với các biện pháp bảo vệ chống lại các hình thức tấn công phổ biến. Để phòng chống SQL Injection, tất cả các truy vấn cơ sở dữ liệu phải sử dụng PDO prepared statements với tham số được ràng buộc an toàn. Để ngăn chặn các cuộc tấn công XSS, mọi dữ liệu đầu vào từ người dùng cần được kiểm tra và làm sạch kỹ lưỡng trước khi được xử lý hoặc hiển thị. Hệ thống xác thực người dùng cần được triển khai dựa trên session với các biện pháp bảo vệ như regenerate session ID sau khi đăng nhập và đặt thời gian hết hạn cho session. Cuối cùng, hệ thống cần có cơ chế kiểm soát quyền truy cập (role-based access control) để đảm bảo rằng chỉ những người dùng có quyền hạn phù hợp mới có thể truy cập vào các chức năng quản trị.

1.4. Phạm vi nghiên cứu

Để đảm bảo tính khả thi và hiệu quả trong thời gian thực hiện đề tài, phạm vi nghiên cứu được giới hạn cụ thể như sau.

Về mặt chức năng, hệ thống tập trung vào hai nhóm người dùng chính là khách hàng và quản trị viên. Đối với khách hàng, hệ thống cung cấp các chức năng cốt lõi của một nền tảng thương mại điện tử bao gồm xem danh sách sản phẩm với khả năng lọc theo thương hiệu và danh mục, tìm kiếm sản phẩm theo tên hoặc mô tả, xem thông tin chi tiết của từng sản phẩm kèm theo nhiều hình ảnh, quản lý giỏ hàng với khả năng thêm, xóa và cập nhật số lượng sản phẩm, đặt hàng và theo dõi trạng thái đơn hàng, so sánh các sản phẩm khác nhau để đưa ra quyết định mua hàng tốt hơn, và đặc biệt là tương tác với chatbot AI để được tư vấn về sản phẩm. Đối với quản trị viên, hệ thống cung cấp các công cụ để quản lý toàn bộ hoạt động kinh doanh thông qua trang quản trị, bao gồm quản lý sản phẩm với khả năng thêm, sửa, xóa và upload nhiều hình ảnh cho mỗi sản phẩm, quản lý danh mục và thương hiệu, xem và cập nhật trạng thái đơn hàng, và theo dõi thống kê tổng quan về doanh thu và số lượng đơn hàng.

Về mặt công nghệ, hệ thống được xây dựng dựa trên các công nghệ phổ biến và ổn định. Phía backend sử dụng ngôn ngữ lập trình PHP phiên bản 8.2 trở lên để xử lý logic nghiệp vụ và giao tiếp với cơ sở dữ liệu. Hệ quản trị cơ sở dữ liệu MySQL phiên bản 8.0 trở lên được sử dụng để lưu trữ tất cả thông tin về sản phẩm, đơn hàng, người dùng và các dữ liệu liên quan khác. Phía frontend được phát triển bằng

HTML5, CSS3 và JavaScript ES6+ để tạo ra giao diện người dùng hiện đại và tương tác. Framework Bootstrap 5 được sử dụng để đảm bảo giao diện responsive và thống nhất trên các thiết bị. Để xây dựng chatbot AI, hệ thống tích hợp với Google Gemini API phiên bản 2.5 Flash, một mô hình ngôn ngữ lớn tiên tiến với chi phí hợp lý và tốc độ phản hồi nhanh.

Về mặt dữ liệu, nghiên cứu tập trung vào lĩnh vực kinh doanh điện thoại thông minh và máy tính bảng của các thương hiệu phổ biến như Apple, Samsung, Xiaomi, Oppo Cơ sở dữ liệu bao gồm thông tin về các sản phẩm như tên, mô tả, giá cả, số lượng tồn kho, thông số kỹ thuật cơ bản, và nhiều hình ảnh minh họa. Chatbot được thiết kế để trả lời các loại câu hỏi phổ biến nhất từ khách hàng, bao gồm câu hỏi về giá và tình trạng có hàng của sản phẩm cụ thể, so sánh giữa các sản phẩm khác nhau, thông số kỹ thuật và tính năng của sản phẩm, gợi ý sản phẩm dựa trên ngân sách hoặc nhu cầu sử dụng, cũng như thông tin về chính sách bảo hành và đổi trả.

1.5. Phương pháp nghiên cứu

Để đạt được các mục tiêu đã đề ra, đề tài áp dụng phương pháp nghiên cứu kết hợp giữa lý thuyết và thực hành, được thực hiện qua các bước cụ thể.

Giai đoạn đầu tiên là nghiên cứu lý thuyết và tổng quan tài liệu. Trong giai đoạn này, tác giả tiến hành thu thập và phân tích các tài liệu học thuật, báo cáo nghiên cứu và bài báo khoa học liên quan đến thương mại điện tử, trí tuệ nhân tạo, và đặc biệt là kiến trúc RAG. Việc nghiên cứu các hệ thống thương mại điện tử hiện có như Shopee, Lazada, và các trang thương mại điện tử chuyên về điện thoại như Thế Giới Di Động, FPT Shop giúp tác giả hiểu rõ các chức năng cơ bản và mong đợi của người dùng. Đồng thời, việc tìm hiểu về các giải pháp chatbot AI đang được sử dụng trong thực tế cũng như các tài liệu kỹ thuật về Google Gemini API và các best practices trong việc tối ưu hóa chi phí API cung cấp nền tảng kiến thức vững chắc cho việc thiết kế hệ thống.

Giai đoạn thứ hai là phân tích và thiết kế hệ thống. Dựa trên kết quả nghiên cứu lý thuyết, tác giả tiến hành phân tích các yêu cầu chức năng và phi chức năng của hệ thống một cách chi tiết. Mô hình hóa hệ thống được thực hiện thông qua các sơ đồ use case để mô tả các tương tác giữa người dùng và hệ thống, sơ đồ hoạt động (activity diagram) để thể hiện quy trình nghiệp vụ, và sơ đồ tuần tự (sequence diagram) để minh họa luồng tương tác giữa các thành phần. Thiết kế cơ sở dữ liệu được thực hiện cẩn thận với sơ đồ ERD (Entity-Relationship Diagram) để đảm bảo cấu trúc dữ liệu hợp lý, chuẩn hóa và có thể mở rộng trong tương lai. Đặc biệt, kiến

trúc RAG cho chatbot được thiết kế chi tiết với năm bước chính là cache, phát hiện ý định, truy xuất dữ liệu, tăng cường thông tin và sinh câu trả lời, nhằm đảm bảo tính chính xác và hiệu quả.

Giai đoạn thứ ba là triển khai và xây dựng hệ thống. Quá trình triển khai được thực hiện theo phương pháp Agile với các sprint ngắn, cho phép linh hoạt trong việc điều chỉnh và cải tiến hệ thống. Việc xây dựng được chia thành các module độc lập bao gồm module cơ sở dữ liệu với việc tạo các bảng và thiết lập các ràng buộc, module backend với việc phát triển các API và logic xử lý, module frontend với việc xây dựng giao diện người dùng, và module chatbot AI với việc tích hợp Google Gemini API và triển khai kiến trúc RAG. Mã nguồn được quản lý thông qua hệ thống kiểm soát phiên bản để đảm bảo tính nhất quán và khả năng theo dõi các thay đổi.

Giai đoạn thứ tư là kiểm thử và tối ưu hóa. Hệ thống được kiểm thử toàn diện qua nhiều cấp độ khác nhau. Unit testing được thực hiện để kiểm tra từng chức năng nhỏ nhất của hệ thống, đảm bảo mỗi module hoạt động đúng như mong đợi. Integration testing được tiến hành để kiểm tra sự tương tác giữa các module khác nhau, đảm bảo chúng hoạt động ăn khớp với nhau. Performance testing được thực hiện để đo lường thời gian tải trang, thời gian phản hồi của chatbot, và khả năng xử lý đồng thời nhiều người dùng. Security testing bao gồm việc kiểm tra các lỗ hổng bảo mật phổ biến như SQL Injection và XSS. Dựa trên kết quả kiểm thử, hệ thống được tối ưu hóa về mặt hiệu suất thông qua việc cải thiện câu truy vấn database, tối ưu hóa code và caching, cũng như giảm chi phí API bằng cách áp dụng smart filtering và caching response.

Giai đoạn cuối cùng là đánh giá và hoàn thiện. Kết quả của hệ thống được đánh giá dựa trên nhiều tiêu chí khác nhau bao gồm tính đầy đủ của các chức năng đã đề ra, hiệu suất hệ thống về thời gian phản hồi và khả năng chịu tải, độ chính xác của chatbot trong việc trả lời câu hỏi và cung cấp thông tin sản phẩm, chi phí vận hành thực tế so với mục tiêu, và mức độ an toàn bảo mật. Tài liệu hệ thống được hoàn thiện bao gồm tài liệu kỹ thuật mô tả kiến trúc và cách thức hoạt động của từng module, hướng dẫn sử dụng cho người dùng cuối và quản trị viên, và hướng dẫn triển khai và bảo trì hệ thống cho các lập trình viên trong tương lai.

1.6. Ý nghĩa của đề tài

Đề tài mang lại những ý nghĩa quan trọng cả về mặt lý thuyết lẫn thực tiễn trong lĩnh vực thương mại điện tử và ứng dụng trí tuệ nhân tạo.

Về mặt lý thuyết, đề tài đóng góp vào việc nghiên cứu và triển khai thực tế kiến trúc RAG trong bối cảnh thương mại điện tử tại Việt Nam. Mặc dù RAG là một kiến trúc đã được nghiên cứu rộng rãi trên thế giới, việc ứng dụng nó vào hệ thống thương mại điện tử với yêu cầu về độ chính xác thông tin thời gian thực và tối ưu hóa chi phí vẫn còn nhiều thách thức. Đề tài cung cấp một case study cụ thể về cách thiết kế và triển khai một chatbot AI hiệu quả với chi phí hợp lý cho doanh nghiệp vừa và nhỏ. Hơn nữa, việc phân tích chi tiết các kỹ thuật tối ưu hóa như smart filtering, caching và rate limiting trong bối cảnh sử dụng API AI thương mại mang lại những insights hữu ích cho cộng đồng nghiên cứu và phát triển.

Về mặt thực tiễn, hệ thống Wireless World được xây dựng có thể được sử dụng trực tiếp như một nền tảng thương mại điện tử hoàn chỉnh cho các doanh nghiệp kinh doanh điện thoại và thiết bị di động. Với đầy đủ các chức năng cần thiết từ quản lý sản phẩm, xử lý đơn hàng đến tư vấn khách hàng tự động, hệ thống giúp giảm thiểu chi phí vận hành đáng kể so với việc thuê nhân viên chăm sóc khách hàng toàn thời gian. Chatbot AI có khả năng phục vụ khách hàng 24/7 với thông tin chính xác và cập nhật, từ đó cải thiện trải nghiệm mua sắm và tăng khả năng chuyển đổi. Đối với các sinh viên và lập trình viên, đề tài cung cấp một mô hình tham khảo hoàn chỉnh về cách xây dựng một hệ thống thương mại điện tử với tích hợp AI, từ giai đoạn phân tích, thiết kế cho đến triển khai và tối ưu hóa.

Đề tài cũng có ý nghĩa trong việc thúc đẩy xu hướng chuyển đổi số cho các doanh nghiệp nhỏ tại Việt Nam. Bằng cách chứng minh rằng việc xây dựng một hệ thống thương mại điện tử với AI chatbot không nhất thiết phải tốn kém và phức tạp, đề tài mở ra cơ hội cho nhiều doanh nghiệp có thể tiếp cận và ứng dụng công nghệ này để nâng cao năng lực cạnh tranh. Hệ thống được thiết kế với kiến trúc rõ ràng, dễ bảo trì và có khả năng mở rộng, cho phép các doanh nghiệp có thể tùy chỉnh và phát triển thêm các tính năng theo nhu cầu cụ thể của mình trong tương lai.

1.7. Cấu trúc khóa luận

Để trình bày một cách có hệ thống và logic các nội dung nghiên cứu, khóa luận được tổ chức thành sáu chương với nội dung cụ thể như sau.

Chương 1 - Giới thiệu: Chương này trình bày bối cảnh và lý do chọn đề tài, làm rõ các vấn đề thực tiễn cần giải quyết trong lĩnh vực thương mại điện tử hiện nay. Các mục tiêu nghiên cứu cụ thể được xác định rõ ràng, cùng với phạm vi nghiên cứu để đảm bảo tính khả thi. Phương pháp nghiên cứu được áp dụng được mô tả chi tiết từ giai đoạn nghiên cứu lý thuyết cho đến triển khai và đánh giá. Cuối cùng, chương này làm rõ ý nghĩa của đề tài đối với cả lý thuyết và thực tiễn.

Chương 2 - Cơ sở lý thuyết: Chương này cung cấp nền tảng kiến thức cần thiết để hiểu rõ các khái niệm và công nghệ được sử dụng trong đề tài. Phần đầu tiên giới thiệu tổng quan về thương mại điện tử, các mô hình kinh doanh phổ biến, và xu hướng phát triển hiện nay. Tiếp theo là phần trình bày về trí tuệ nhân tạo và các mô hình ngôn ngữ lớn, đặc biệt tập trung vào kiến trúc RAG và cách thức hoạt động của nó. Các công nghệ web cơ bản như PHP, MySQL, HTML/CSS/JavaScript cũng được giới thiệu. Phần cuối của chương phân tích các hệ thống thương mại điện tử hiện có và các giải pháp chatbot AI đang được sử dụng, từ đó rút ra những bài học kinh nghiệm cho việc thiết kế hệ thống.

Chương 3 - Phân tích và thiết kế hệ thống: Chương này trình bày chi tiết quá trình phân tích yêu cầu và thiết kế kiến trúc hệ thống. Phần phân tích yêu cầu xác định rõ các chức năng mà hệ thống cần cung cấp cho từng loại người dùng, cũng như các yêu cầu phi chức năng về hiệu suất, bảo mật và khả năng mở rộng. Thiết kế kiến trúc hệ thống áp dụng mô hình 3 tầng (3-tier architecture) với tầng giao diện, tầng xử lý logic và tầng dữ liệu. Thiết kế cơ sở dữ liệu được trình bày qua sơ đồ ERD với mô tả chi tiết về các bảng, thuộc tính và mối quan hệ. Đặc biệt, thiết kế chi tiết kiến trúc RAG cho chatbot với năm bước xử lý được mô tả kỹ lưỡng. Các sơ đồ use case, activity diagram và sequence diagram cũng được sử dụng để minh họa các luồng hoạt động chính của hệ thống.

Chương 4 - Triển khai hệ thống: Chương này mô tả quá trình triển khai thực tế các module của hệ thống. Phần đầu tiên trình bày việc thiết lập môi trường phát triển và các công cụ được sử dụng. Tiếp theo là việc triển khai cơ sở dữ liệu với các đoạn mã SQL để tạo bảng và thiết lập ràng buộc. Module backend được triển khai với các file PHP xử lý logic nghiệp vụ, kết nối database và cung cấp API. Module frontend được phát triển với HTML/CSS/JavaScript để tạo giao diện người dùng. Phần quan trọng nhất là việc triển khai chatbot AI với tích hợp Google Gemini API, xây dựng các bước của kiến trúc RAG, và triển khai các kỹ thuật tối ưu hóa như smart filtering và caching. Mỗi phần đều kèm theo các đoạn mã nguồn minh họa và giải thích chi tiết.

Chương 5 - Kiểm thử và đánh giá: Chương này trình bày các phương pháp kiểm thử được áp dụng và kết quả đánh giá hệ thống. Kiểm thử chức năng được thực hiện để đảm bảo tất cả các tính năng hoạt động đúng như yêu cầu. Kiểm thử hiệu suất đo lường các chỉ số quan trọng như thời gian tải trang, thời gian phản hồi của chatbot, và khả năng xử lý đồng thời. Kiểm thử bảo mật được tiến hành để phát hiện các lỗ hổng tiềm ẩn. Kết quả được phân tích và so sánh với các mục tiêu ban đầu để

đánh giá mức độ thành công của đề tài. Các vấn đề gặp phải trong quá trình triển khai và cách giải quyết cũng được thảo luận.

Chương 6 - Kết luận và hướng phát triển: Chương cuối cùng tổng kết lại các kết quả đã đạt được của đề tài, nhấn mạnh những đóng góp chính về mặt lý thuyết và thực tiễn. Những hạn chế của hệ thống hiện tại được thừa nhận một cách trung thực. Dựa trên đó, các hướng phát triển trong tương lai được đề xuất để hoàn thiện và mở rộng hệ thống, bao gồm việc tích hợp thêm các tính năng như thanh toán trực tuyến, hệ thống đánh giá sản phẩm, chương trình khuyến mãi, và cải thiện khả năng của chatbot AI. Kinh nghiệm rút ra từ quá trình thực hiện đề tài cũng được chia sẻ để làm tài liệu tham khảo cho các nghiên cứu tương tự trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1. Tổng quan về thương mại điện tử

Thương mại điện tử, hay e-commerce, đã trở thành một phần không thể thiếu trong nền kinh tế số hiện đại. Về bản chất, thương mại điện tử là hoạt động mua bán, trao đổi hàng hóa và dịch vụ được thực hiện thông qua môi trường Internet và các phương tiện điện tử khác. Tuy nhiên, trong bối cảnh phát triển hiện nay, e-commerce đã vượt xa khái niệm đơn thuần về việc chuyển đổi các giao dịch truyền thống sang môi trường số, mà còn bao trùm toàn bộ chuỗi giá trị từ marketing, quảng cáo sản phẩm, quá trình bán hàng, thanh toán, giao nhận hàng hóa, cho đến các dịch vụ chăm sóc và hỗ trợ khách hàng sau bán.

Dựa trên đối tượng tham gia giao dịch, thương mại điện tử được phân loại thành nhiều mô hình khác nhau. Mô hình B2C (Business-to-Consumer) là mô hình phổ biến nhất, trong đó doanh nghiệp bán sản phẩm hoặc dịch vụ trực tiếp cho người tiêu dùng cá nhân. Mô hình B2B (Business-to-Business) diễn ra giữa các doanh nghiệp với nhau, thường liên quan đến các giao dịch với giá trị lớn và quy trình phức tạp hơn. Mô hình C2C (Consumer-to-Consumer) cho phép người tiêu dùng bán hàng cho nhau thông qua các nền tảng trung gian như các trang marketplace. Cuối cùng là mô hình C2B (Consumer-to-Business), nơi người tiêu dùng cung cấp sản phẩm hoặc dịch vụ cho doanh nghiệp, ví dụ như freelancer cung cấp dịch vụ thiết kế cho công ty. Đề tài Wireless World được xây dựng theo mô hình B2C, tập trung vào việc kết nối doanh nghiệp kinh doanh điện thoại và máy tính bảng với người tiêu dùng cuối.

Kiến trúc của một hệ thống thương mại điện tử hiện đại thường được tổ chức theo mô hình ba tầng để đảm bảo tính module hóa, dễ bảo trì và khả năng mở rộng. Tầng giao diện, hay presentation layer, chịu trách nhiệm tương tác trực tiếp với người dùng thông qua các trang web như trang chủ hiển thị sản phẩm nổi bật, trang danh mục cho phép lọc và tìm kiếm sản phẩm, trang chi tiết sản phẩm với đầy đủ thông số kỹ thuật và hình ảnh, trang giỏ hàng và quy trình thanh toán, cũng như trang tài khoản cá nhân cho người dùng theo dõi lịch sử mua hàng. Tầng logic nghiệp vụ, hay business logic layer, xử lý các tác vụ quan trọng như xác thực và phân quyền người dùng, quản lý catalog sản phẩm bao gồm danh mục và thương hiệu, xử lý đơn hàng từ lúc tạo đến khi hoàn thành, quản lý kho hàng để theo dõi số lượng tồn kho, và tích hợp với các dịch vụ bên thứ ba như cổng thanh toán và đơn vị vận chuyển. Tầng dữ liệu, hay data layer, đảm nhiệm việc lưu trữ bền vững tất cả thông tin quan

trọng bao gồm thông tin người dùng với dữ liệu xác thực và hồ sơ cá nhân, catalog sản phẩm với hình ảnh và thông số chi tiết, đơn hàng và các chi tiết giao dịch, cũng như lịch sử hoạt động của khách hàng phục vụ cho phân tích và cải thiện dịch vụ.

Các nền tảng thương mại điện tử hiện nay đang phải đối mặt với nhiều thách thức quan trọng cần được giải quyết. Về trải nghiệm người dùng, khách hàng ngày càng đòi hỏi cao với kỳ vọng trang web phải tải nhanh trong vòng ba giây, giao diện phải thân thiện và hoạt động mượt mà trên mọi thiết bị từ máy tính để bàn đến điện thoại di động, và quy trình thanh toán phải được tối giản với số bước ít nhất có thể. Trong lĩnh vực chăm sóc khách hàng, doanh nghiệp cần cung cấp dịch vụ hỗ trợ liên tục suốt hai mươi bốn giờ mỗi ngày và bảy ngày mỗi tuần, nhưng chi phí để duy trì đội ngũ nhân viên làm việc toàn thời gian là rất cao. Thời gian phản hồi chậm có thể dẫn đến việc mất khách hàng, trong khi khối lượng câu hỏi từ người dùng ngày càng lớn khiến việc quản lý trở nên khó khăn. Vấn đề bảo mật cũng là một mối quan ngại lớn, vì hệ thống cần bảo vệ thông tin cá nhân và dữ liệu thanh toán của khách hàng, chống lại các cuộc tấn công mạng như SQL Injection, Cross-Site Scripting và DDoS, đồng thời tuân thủ các quy định nghiêm ngặt về bảo vệ dữ liệu. Cuối cùng, trong một thị trường cạnh tranh gay gắt với hàng loạt đối thủ, việc khách hàng có thể dễ dàng so sánh giá giữa các nền tảng và chi phí marketing ngày càng tăng cao đã tạo ra áp lực lớn đối với các doanh nghiệp.

2.2. Công nghệ RAG (Retrieval-Augmented Generation)

Công nghệ RAG, viết tắt của Retrieval-Augmented Generation, đại diện cho một bước tiến quan trọng trong lĩnh vực trí tuệ nhân tạo ứng dụng. Về bản chất, RAG là một kiến trúc AI tiên tiến kết hợp hai khả năng then chốt: khả năng truy xuất thông tin từ nguồn dữ liệu bên ngoài và khả năng sinh văn bản tự nhiên từ các mô hình ngôn ngữ lớn. Thành phần retrieval, hay truy xuất, có nhiệm vụ tìm kiếm và lấy ra những thông tin liên quan nhất từ các nguồn dữ liệu như cơ sở dữ liệu, APIs, hoặc tài liệu văn bản. Thành phần generation, hay sinh văn bản, sử dụng các mô hình ngôn ngữ lớn như GPT hay Gemini để tạo ra câu trả lời tự nhiên dựa trên thông tin đã được truy xuất. Điểm khác biệt cơ bản giữa RAG và các chatbot truyền thống nằm ở chỗ, trong khi chatbot thông thường chỉ dựa vào kiến thức tĩnh được huấn luyện trước và không thể cập nhật thông tin mới, RAG có khả năng truy cập dữ liệu động từ các nguồn thời gian thực, giúp câu trả lời luôn chính xác và cập nhật.

Quy trình hoạt động của RAG có thể được mô tả qua bốn bước chính, tạo nên một chu trình xử lý thông tin hiệu quả. Bước đầu tiên là tiếp nhận câu hỏi từ người

dùng, ví dụ như "Giá của Samsung Galaxy A56 là bao nhiêu?". Hệ thống nhận câu hỏi này và tiến hành xử lý ngôn ngữ tự nhiên để hiểu ý định. Bước thứ hai là giai đoạn truy xuất dữ liệu, nơi hệ thống phân tích câu hỏi để xác định rõ intent của người dùng, trong trường hợp này là tìm kiếm giá sản phẩm. Sau đó, một truy vấn được thực hiện vào cơ sở dữ liệu để lấy thông tin về sản phẩm Samsung Galaxy A56, trả về kết quả bao gồm tên sản phẩm, giá bán và số lượng tồn kho. Bước thứ ba là tăng cường context, hay augmentation, trong đó hệ thống kết hợp dữ liệu vừa truy xuất được với câu hỏi ban đầu để tạo thành một prompt đầy đủ. Prompt này có dạng "Dựa trên catalog: Samsung Galaxy A56 có giá \$449.99 và còn 15 chiếc trong kho. Hãy trả lời câu hỏi: Giá của Samsung Galaxy A56 là bao nhiêu?". Bước cuối cùng là sinh câu trả lời, nơi prompt được gửi đến mô hình ngôn ngữ lớn như Google Gemini, và mô hình tạo ra câu trả lời tự nhiên, thân thiện như "Samsung Galaxy A56 có giá \$449.99 và hiện đang còn hàng với 15 chiếc trong kho của chúng tôi".

Khi so sánh RAG với các chatbot truyền thống, những ưu điểm vượt trội của RAG trở nên rõ ràng. Chatbot truyền thống, được xây dựng dựa trên các quy tắc định sẵn hoặc phát hiện ý định đơn giản, tuy có ưu điểm là đơn giản, dễ triển khai và chi phí thấp, nhưng lại bị giới hạn nghiêm trọng về khả năng. Chúng chỉ có thể trả lời các câu hỏi đã được lập trình trước, không thể cập nhật thông tin theo thời gian thực như giá cả hay tồn kho, và không có khả năng hiểu ngữ cảnh phức tạp. Dữ liệu trong các chatbot này là tĩnh và thiếu linh hoạt. Ngược lại, chatbot sử dụng RAG mang lại nhiều lợi thế đáng kể. Chúng có khả năng trả lời chính xác với dữ liệu được lấy trực tiếp từ cơ sở dữ liệu trong thời gian thực, hiểu được ngữ cảnh và xử lý các câu hỏi phức tạp, tự động cập nhật khi có thay đổi trong database mà không cần lập trình lại, tạo ra câu trả lời tự nhiên giống như cách con người giao tiếp, và có thể xử lý cả những câu hỏi chưa từng gặp trước đây. Tuy nhiên, RAG cũng có những nhược điểm nhất định như chi phí sử dụng API cao hơn, cần phải tối ưu hóa cẩn thận để giảm số lượng token được xử lý, và phụ thuộc rất nhiều vào chất lượng của dữ liệu nguồn.

Trong bối cảnh ứng dụng vào thương mại điện tử, RAG mang lại giá trị đặc biệt quan trọng. Về mặt hỗ trợ khách hàng tự động, RAG cho phép trả lời các câu hỏi về sản phẩm với thông tin được lấy trực tiếp và chính xác từ cơ sở dữ liệu, cập nhật liên tục về giá cả, chương trình khuyến mãi và tình trạng tồn kho theo thời gian thực, và đưa ra các gợi ý sản phẩm phù hợp dựa trên nhu cầu cụ thể của từng khách hàng. Về khía cạnh giảm chi phí vận hành, việc sử dụng RAG giúp doanh nghiệp không cần phải duy trì một đội ngũ nhân viên chăm sóc khách hàng làm việc hai mươi bốn giờ mỗi ngày, có khả năng xử lý hàng nghìn câu hỏi cùng một lúc, và giảm thời gian phản hồi từ vài phút xuống chỉ còn vài giây. Cuối cùng, về cải thiện trải

nghiệm người dùng, khách hàng có thể nhận được câu trả lời ngay lập tức mà không cần chờ đợi, giao tiếp với hệ thống một cách tự nhiên bằng ngôn ngữ thường ngày, và không bị giới hạn bởi giờ làm việc hay múi giờ.

2.3. Các công nghệ triển khai

2.3.1. PHP và quản lý cơ sở dữ liệu

PHP, viết tắt của Hypertext Preprocessor, là ngôn ngữ lập trình server-side được sử dụng rộng rãi nhất trong phát triển web hiện nay. Theo thống kê của W3Techs năm 2025, PHP được sử dụng bởi hơn 77.4% các website trên toàn cầu, chứng tỏ sức ảnh hưởng và độ tin cậy của ngôn ngữ này. Trong bối cảnh dự án Wireless World, PHP được lựa chọn do nhiều lý do quan trọng. Đầu tiên, PHP có cú pháp đơn giản, dễ học và dễ triển khai, phù hợp cho cả người mới bắt đầu lẫn lập trình viên có kinh nghiệm. Thứ hai, PHP có khả năng hỗ trợ tuyệt vời cho các hệ quản trị cơ sở dữ liệu phổ biến như MySQL và PostgreSQL. Thứ ba, cộng đồng PHP rất lớn mạnh với hàng nghìn thư viện và framework sẵn có, giúp giải quyết nhanh chóng các vấn đề thường gặp. Thứ tư, chi phí hosting cho PHP rất thấp vì hầu hết các web server đều hỗ trợ sẵn ngôn ngữ này. Cuối cùng, PHP tích hợp tự nhiên và mượt mà với HTML, CSS và JavaScript, tạo nên một môi trường phát triển thống nhất.

Một trong những tính năng quan trọng nhất của PHP được sử dụng trong dự án là PDO, viết tắt của PHP Data Objects. PDO cung cấp một lớp truy cập cơ sở dữ liệu thống nhất, cho phép làm việc với nhiều loại database khác nhau thông qua cùng một interface. Điểm mạnh lớn nhất của PDO là khả năng hỗ trợ prepared statements, một kỹ thuật quan trọng để chống lại các cuộc tấn công SQL Injection. Với prepared statements, các tham số được tách biệt khỏi câu lệnh SQL và được xử lý an toàn, ngăn chặn khả năng injection code độc hại. Ngoài ra, PDO còn có cơ chế xử lý lỗi tốt hơn thông qua exceptions, giúp phát hiện và khắc phục sự cố dễ dàng hơn. Một ví dụ điển hình về cách sử dụng PDO trong dự án là khi truy vấn thông tin sản phẩm: hệ thống chuẩn bị câu lệnh SQL với placeholder, sau đó thực thi với tham số `product_id` được bind an toàn, và cuối cùng lấy kết quả dưới dạng mảng kết hợp.

Session management, hay quản lý phiên làm việc, là một khía cạnh quan trọng khác của PHP trong ứng dụng web. Session cho phép lưu trữ thông tin người dùng giữa các HTTP requests khác nhau, vốn là stateless bản chất. Trong PHP, session được khởi tạo bằng hàm `session_start()` và dữ liệu được lưu trong biến toàn cục `$_SESSION`. Cơ chế này có nhiều ứng dụng thiết yếu trong hệ thống Wireless

World, bao gồm việc xác thực người dùng để duy trì trạng thái đăng nhập, lưu trữ giỏ hàng cho cả khách vắng lần người dùng đã đăng ký, và cache dữ liệu chatbot để cải thiện hiệu suất phản hồi.

MySQL, một hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, đóng vai trò then chốt trong việc lưu trữ và quản lý dữ liệu của hệ thống. MySQL được lựa chọn nhờ các đặc điểm ưu việt như hiệu suất cao ngay cả khi xử lý khối lượng dữ liệu lớn, hỗ trợ đầy đủ các thuộc tính ACID đảm bảo tính toàn vẹn dữ liệu, khả năng mở rộng tốt để đáp ứng nhu cầu tăng trưởng, hệ thống bảo mật với phân quyền chi tiết, và Event Scheduler cho phép tự động hóa các tác vụ định kỳ.

Các tính năng quan trọng của MySQL được áp dụng trong dự án bao gồm Foreign Keys, Indexes, Event Scheduler và Transactions. Foreign Keys, hay khóa ngoại, đảm bảo tính toàn vẹn tham chiếu giữa các bảng trong cơ sở dữ liệu. Ví dụ, khi một sản phẩm bị xóa, tất cả các hình ảnh liên quan trong bảng `product_images` cũng tự động được xóa theo nhờ `CASCADE DELETE`, tránh tình trạng dữ liệu mồ côi. Indexes, hay chỉ mục, giúp tăng tốc độ truy vấn dữ liệu đáng kể. Primary key tự động tạo index, và các index bổ sung được thiết lập trên các cột thường xuyên được tìm kiếm như tên sản phẩm hay `category_id`. Event Scheduler cho phép tự động hóa các tác vụ bảo trì như dọn dẹp lịch sử chat cũ hơn ba mươi ngày, chạy theo lịch định kỳ hàng ngày hoặc hàng giờ. Transactions đảm bảo tính nhất quán khi có nhiều thao tác cần được thực hiện như một đơn vị công việc, ví dụ khi xử lý đơn hàng cần đồng thời trừ số lượng trong kho, tạo bản ghi order và tạo các order items, với khả năng rollback nếu có bất kỳ lỗi nào xảy ra.

2.3.2. Framework và công nghệ frontend

Bootstrap 5 là framework CSS front-end được sử dụng rộng rãi nhất hiện nay, cung cấp một bộ công cụ mạnh mẽ để xây dựng giao diện web responsive và chuyên nghiệp. Hệ thống lưới, hay Grid System, của Bootstrap dựa trên mô hình 12 cột linh hoạt với các breakpoints responsive cho nhiều kích thước màn hình khác nhau từ điện thoại di động nhỏ đến màn hình desktop lớn. Điều này cho phép giao diện tự động điều chỉnh và hiển thị tối ưu trên mọi thiết bị.

Bootstrap cung cấp một bộ sưu tập phong phú các components, hay thành phần UI có sẵn. Navbar tạo ra thanh điều hướng responsive với khả năng tự động thu gọn trên màn hình nhỏ. Cards được sử dụng để hiển thị sản phẩm một cách đẹp mắt với hình ảnh và thông tin. Modals cho phép tạo các popup để xác nhận hành động hoặc hiển thị thông báo. Forms cung cấp các input, select, textarea với

validation tích hợp sẵn. Buttons có nhiều style và size khác nhau phù hợp với từng mục đích sử dụng.

Các utilities, hay tiện ích CSS, của Bootstrap giúp điều chỉnh nhanh chóng các thuộc tính mà không cần viết CSS tùy chỉnh. Spacing utilities như m-3 cho margin và p-4 cho padding giúp tạo khoảng cách nhất quán. Color utilities như text-primary và bg-success áp dụng màu sắc theo theme. Display utilities như d-flex, d-none, d-md-block kiểm soát hiển thị phần tử. Typography utilities như fs-1 và fw-bold điều chỉnh font size và font weight. Bootstrap cũng tích hợp các JavaScript plugins hữu ích như dropdown menu, collapse content, toast notifications và carousel cho trình chiếu ảnh.

Lợi ích của Bootstrap trong dự án Wireless World rất rõ ràng: tiết kiệm đáng kể thời gian phát triển giao diện, đảm bảo tương thích tốt trên mọi trình duyệt hiện đại, tự động responsive trên mọi kích thước thiết bị, và tạo ra giao diện nhất quán, chuyên nghiệp mà không cần thiết kế từ đầu.

JavaScript ES6+ là ngôn ngữ lập trình client-side hiện đại cho phép tạo ra các tương tác động và phong phú trên trang web. Các tính năng của ES6 và các phiên bản sau đó đã cải thiện đáng kể khả năng và sự thuận tiện của JavaScript. Arrow functions cung cấp cú pháp ngắn gọn và xử lý this context tốt hơn, đặc biệt phù hợp cho các callback functions. Async/Await giúp xử lý các tác vụ bất đồng bộ một cách dễ đọc và dễ bảo trì hơn so với Promises truyền thống hoặc callback hell. Template literals cho phép nhúng biến trực tiếp vào chuỗi và hỗ trợ chuỗi nhiều dòng, rất hữu ích khi tạo HTML động. Destructuring giúp giải cấu trúc object và array một cách ngắn gọn, làm code dễ đọc hơn.

Fetch API là một tính năng quan trọng thay thế cho XMLHttpRequest cũ, cung cấp interface hiện đại để thực hiện các HTTP requests. Fetch trả về Promise, cho phép xử lý bất đồng bộ một cách uyển chuyển, và được sử dụng rộng rãi trong chatbot để giao tiếp với server. DOM Manipulation, hay thao tác với Document Object Model, cho phép JavaScript tương tác và thay đổi nội dung trang web động. Các phương thức như querySelector và getElementById giúp chọn các phần tử HTML, addEventListener xử lý các sự kiện từ người dùng,.classList.add và remove thêm hoặc xóa CSS classes, và innerHTML cùng textContent thay đổi nội dung hiển thị.

2.3.3. Google Gemini API

Google Gemini là dòng mô hình AI multimodal mới nhất của Google, được ra mắt vào cuối năm 2024 và đại diện cho thế hệ tiếp theo của công nghệ trí tuệ nhân

tạo. Gemini được phát triển với ba phiên bản chính phục vụ các nhu cầu khác nhau. Gemini Ultra là phiên bản mạnh mẽ nhất, được thiết kế để xử lý các tác vụ phức tạp nhất, phù hợp cho nghiên cứu và các ứng dụng doanh nghiệp lớn nhưng có chi phí cao nhất. Gemini Pro cân bằng giữa hiệu suất và chi phí, phù hợp cho các ứng dụng production thực tế với khả năng xử lý text, code và images. Gemini 2.5 Flash, phiên bản được sử dụng trong dự án này, là phiên bản mới nhất tối ưu hóa cho tốc độ và hiệu quả chi phí.

Gemini 2.5 Flash có nhiều ưu điểm vượt trội khiến nó trở thành lựa chọn lý tưởng cho chatbot e-commerce. Context window lên đến một triệu tokens cho phép xử lý lượng thông tin cực lớn trong một lần request. Tốc độ phản hồi rất nhanh, thường chỉ từ một đến hai giây, đáp ứng yêu cầu về trải nghiệm người dùng. Đặc biệt, Google cung cấp free tier không giới hạn số lượng requests mỗi ngày, giúp giảm chi phí vận hành xuống gần như bằng không cho các dự án quy mô vừa và nhỏ.

Các tính năng chính của Gemini API bao gồm text generation với khả năng tạo câu trả lời tự nhiên từ prompt, hỗ trợ context dài lên đến 32,000 tokens cho mỗi request, và temperature control để điều chỉnh mức độ sáng tạo hoặc nhất quán của câu trả lời. API Integration được thiết kế theo chuẩn RESTful, dễ dàng tích hợp, với authentication đơn giản qua API key và hỗ trợ streaming responses để cải thiện trải nghiệm người dùng.

Các parameters quan trọng khi sử dụng Gemini API cần được hiểu rõ để tối ưu hóa kết quả. Temperature là tham số kiểm soát mức độ ngẫu nhiên trong câu trả lời, với giá trị từ 0.0 đến 1.0, trong đó 0.0 tạo ra câu trả lời nhất quán và dự đoán được, còn 1.0 tạo ra câu trả lời sáng tạo và đa dạng hơn. Dự án sử dụng giá trị 0.2 để đảm bảo tính nhất quán trong các câu trả lời về sản phẩm. MaxOutputTokens giới hạn độ dài của câu trả lời, được đặt ở mức 1000 tokens trong dự án để kiểm soát chi phí và thời gian phản hồi. TopK và topP là các tham số điều chỉnh độ đa dạng của câu trả lời, với topP=0.9 nghĩa là chỉ chọn các từ có tổng xác suất đạt 90%.

Về mặt pricing, Gemini 2.5 Flash có lợi thế cạnh tranh vượt trội. Với free tier, cả input và output đều không tốn phí và không giới hạn số lượng requests. Ngay cả khi chuyển sang paid tier sau khi vượt quota miễn phí, chi phí chỉ là \$0.075 cho input và \$0.30 cho output trên mỗi một triệu tokens. Khi so sánh với các đối thủ cạnh tranh, sự khác biệt rất rõ ràng: GPT-4 Turbo có giá \$10/\$30 per million tokens, đắt hơn hơn một trăm lần; Claude 3 Sonnet có giá \$3/\$15, đắt hơn bốn mươi lần; và ngay cả GPT-3.5 Turbo với giá \$0.5/\$1.5 vẫn đắt hơn năm lần. Điều này khiến

Gemini 2.5 Flash trở thành lựa chọn tối ưu nhất về cả chi phí lẫn hiệu suất cho các chatbot thương mại điện tử.

2.4. Khảo sát các hệ thống hiện có

Để hiểu rõ hơn về bối cảnh thương mại điện tử hiện tại và rút ra các bài học kinh nghiệm, nghiên cứu này đã tiến hành phân tích một số hệ thống tiêu biểu trên thị trường. Shopee, với hơn một trăm triệu người dùng, là nền tảng thương mại điện tử lớn nhất khu vực Đông Nam Á. Shopee nổi bật với nhiều tính năng độc đáo như khả năng chat trực tiếp với người bán, livestreaming bán hàng tạo tính tương tác cao, hệ thống Shopee Coins và gamification để khuyến khích người dùng, các chương trình flash sale và voucher hấp dẫn, cũng như Shopee Pay được tích hợp ngay trong app. Chatbot của Shopee có khả năng tự động trả lời các câu hỏi thường gặp, hỗ trợ tracking đơn hàng, gợi ý sản phẩm dựa trên lịch sử mua hàng và tích hợp với hệ thống CRM. Tuy nhiên, điểm mạnh lớn nhất của Shopee nằm ở quy mô khổng lồ với khả năng xử lý hàng triệu giao dịch mỗi ngày, hệ thống giao hàng riêng SPX và chiến lược marketing mạnh mẽ. Mặt yếu là chatbot của họ vẫn còn hạn chế, chủ yếu dựa trên quy tắc, và có quá nhiều thông báo gây spam người dùng, với giao diện đôi khi phức tạp và khó sử dụng.

Thế Giới Di Động đại diện cho mô hình bán lẻ truyền thống kết hợp online, là chuỗi cửa hàng điện thoại lớn nhất Việt Nam với hơn một nghìn một trăm cửa hàng. Website của họ cung cấp tính năng so sánh chi tiết thông số kỹ thuật các sản phẩm, đánh giá từ khách hàng thực tế, video review chuyên sâu, mô hình "Mua Online - Nhận tại cửa hàng" tiện lợi, và dịch vụ bảo hành tận nơi. Chatbot hỗ trợ chủ yếu qua Facebook Messenger, tích hợp với tổng đài và cung cấp tư vấn sản phẩm cơ bản. Điểm mạnh của Thế Giới Di Động là uy tín cao được xây dựng qua nhiều năm, hệ thống cửa hàng phủ rộng khắp cả nước, dịch vụ sau bán hàng tốt và giá cả cạnh tranh. Tuy nhiên, chatbot của họ chưa thực sự thông minh, chủ yếu chuyển sang tổng đài khi gặp câu hỏi phức tạp, website chưa được tối ưu tốc độ tải trang, và chưa có tính năng recommendation dựa trên AI.

Amazon, với doanh thu 575 tỷ đô la năm 2023, là tập đoàn thương mại điện tử lớn nhất thế giới và tiên phong trong việc ứng dụng AI. Tích hợp Alexa cho phép khách hàng mua hàng bằng giọng nói, tracking đơn hàng hands-free và nhận gợi ý sản phẩm thông minh. Hệ thống recommendation của Amazon, với câu nói nổi tiếng "Customers who bought this also bought...", sử dụng machine learning phân tích hành vi mua hàng để cá nhân hóa trang chủ cho từng người dùng. Amazon Q, chatbot AI dành cho doanh nghiệp, tích hợp với AWS và xử lý các câu hỏi phức tạp về sản

phẩm. Amazon có AI recommendation cực kỳ chính xác, hệ thống logistics tốt nhất thế giới với Prime delivery một ngày, và liên tục thực hiện A/B testing để tối ưu trải nghiệm người dùng trên nền tảng cloud infrastructure mạnh mẽ AWS. Nhược điểm của Amazon là chi phí vận hành cực kỳ cao, hệ thống phức tạp khó có thể sao chép cho doanh nghiệp nhỏ, và chưa phổ biến tại thị trường Việt Nam.

Từ việc nghiên cứu các hệ thống trên, đồ án rút ra những bài học quan trọng cho việc xây dựng Wireless World. Các nền tảng lớn như Shopee và Amazon đều đầu tư mạnh vào công nghệ AI để cải thiện trải nghiệm khách hàng, chứng tỏ xu hướng này là tất yếu. Tuy nhiên, các giải pháp của họ thường phức tạp và tốn kém, không phù hợp với doanh nghiệp vừa và nhỏ. Điều này tạo ra cơ hội cho một hệ thống tối ưu hóa chi phí mà vẫn đảm bảo chất lượng. Wireless World do đó tập trung vào việc triển khai kiến trúc RAG với Google Gemini API, tận dụng free tier và các kỹ thuật tối ưu hóa token để giảm thiểu chi phí. Hệ thống được thiết kế đơn giản, dễ triển khai và bảo trì, phù hợp với nguồn lực của doanh nghiệp vừa và nhỏ. Việc tập trung vào thị trường ngách điện thoại và máy tính bảng cho phép tối ưu hóa sâu cho lĩnh vực cụ thể thay vì cố gắng phục vụ đa ngành hàng. Các tính năng bảo mật và spam prevention được xây dựng ngay từ đầu thay vì bổ sung sau, đảm bảo hệ thống an toàn và ổn định.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu hệ thống

Quá trình phân tích yêu cầu là bước quan trọng đầu tiên trong việc xây dựng hệ thống Wireless World, giúp xác định rõ ràng các chức năng cần thiết và các tiêu chí phi chức năng mà hệ thống phải đáp ứng. Các yêu cầu được phân loại thành hai nhóm chính: yêu cầu chức năng mô tả những gì hệ thống phải làm được, và yêu cầu phi chức năng mô tả hệ thống phải hoạt động như thế nào về mặt hiệu suất, bảo mật và độ tin cậy.

Yêu cầu chức năng cho Khách hàng:

- **Xem và lọc sản phẩm:** Danh sách sản phẩm với bộ lọc theo thương hiệu (Samsung, Apple, Xiaomi, Oppo, Vivo) và danh mục (Điện thoại, Máy tính bảng)
- **Tìm kiếm:** Tìm kiếm sản phẩm theo tên hoặc mô tả với kết quả nhanh và chính xác
- **Xem chi tiết sản phẩm:** Hiển thị nhiều hình ảnh chất lượng cao, giá bán và giá khuyến mãi, số lượng tồn kho, thông số kỹ thuật (màn hình, RAM, bộ nhớ, camera, pin, hệ điều hành)
- **Quản lý giỏ hàng:** Thêm sản phẩm, cập nhật số lượng, xóa sản phẩm, tính tổng giá trị tự động
- **Đặt hàng:** Quy trình đơn giản với các bước thu thập thông tin giao hàng, xác nhận đơn hàng và lưu trữ
- **So sánh sản phẩm:** Xem cạnh nhau các thông số kỹ thuật để đưa ra quyết định mua hàng
- **AI Chatbot:**
 - Trả lời câu hỏi về giá cả và tồn kho theo thời gian thực
 - So sánh sản phẩm và cung cấp thông số kỹ thuật
 - Gợi ý sản phẩm phù hợp với ngân sách và nhu cầu
 - Trả lời câu hỏi về chính sách bảo hành và đổi trả

Yêu cầu chức năng cho Quản trị viên:

- Quản lý sản phẩm:

- Thêm sản phẩm mới với upload nhiều hình ảnh và chọn hình đại diện
- Chỉnh sửa thông tin sản phẩm hiện có
- Xóa sản phẩm cùng tất cả hình ảnh liên quan
- Xem danh sách sản phẩm với phân trang

- Quản lý danh mục và thương hiệu: Thêm, sửa, xóa danh mục và thương hiệu

- Quản lý đơn hàng: Xem danh sách, xem chi tiết, cập nhật trạng thái (pending → delivered)

- Dashboard: Hiển thị thống kê tổng quan (doanh thu, số đơn hàng, số sản phẩm, số người dùng)

- Quản lý người dùng: Xem danh sách và thông tin chi tiết các tài khoản

Yêu cầu phi chức năng:

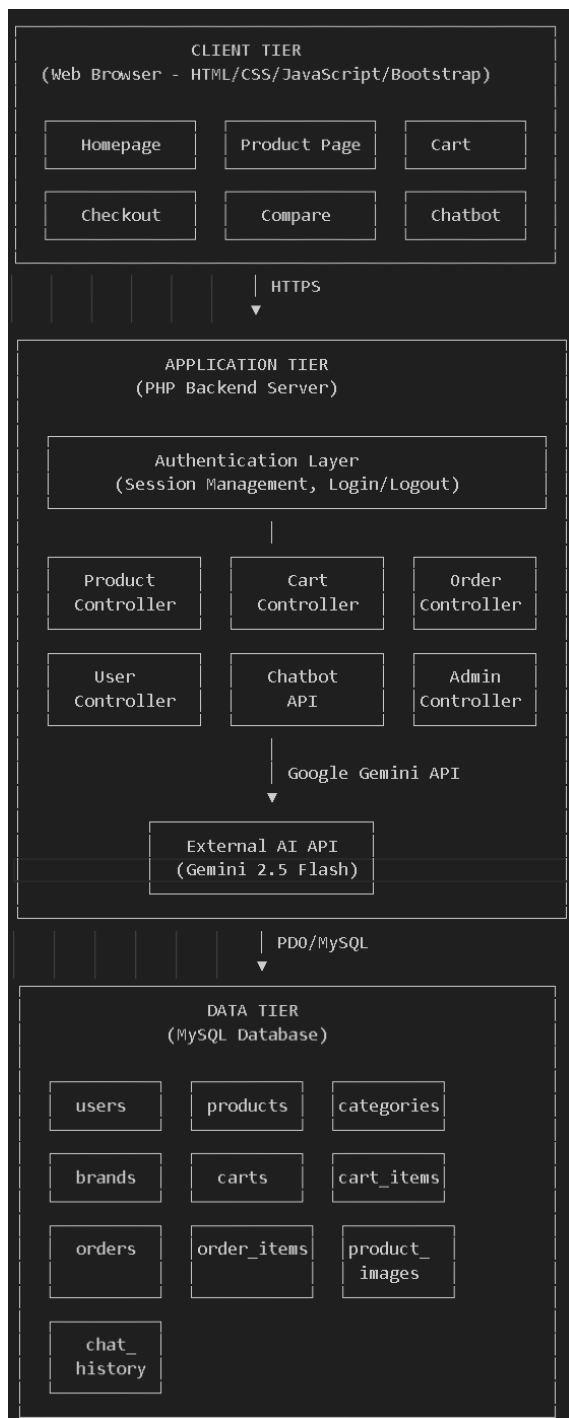
Danh mục	Yêu cầu	Chi tiết / Chỉ số kỹ thuật
Hiệu suất	Thời gian phản hồi chatbot	1–3 giây
	Thời gian tải trang	< 3 giây
	Cache sản phẩm	5 phút
	Tối ưu token usage	Giảm 60–70% thông qua smart filtering
Bảo mật	Mã hóa mật khẩu	Password hashing (Bcrypt/Argon2)
	Phòng chống SQL Injection	Sử dụng PDO Prepared Statements
	Phòng chống XSS	Input sanitization (Lọc dữ liệu đầu vào)
	Xác thực	Session-based với cơ chế regenerate ID
	Rate limiting	Cooldown 3 giây giữa các tin nhắn
	Message quota	20 tin/giờ (guest), 50 tin/giờ (user)
Khả dụng	Uptime chatbot	Hoạt động 24/7 với độ sẵn sàng $\geq 99\%$
	Error handling	Graceful fallback (Xử lý lỗi mượt mà) khi API fails

Mở rộng	Kiến trúc hệ thống	Mô hình 3-tier (3 tầng) dễ mở rộng từng phần
	Cơ sở dữ liệu	Indexing tối ưu cho các truy vấn
	Cache	Session (có khả năng chuyển đổi sang Redis)
Sử dụng	Responsive	Tương thích Mobile, Tablet, Desktop
	UI/UX	Giao diện đơn giản, trực quan
	Thời gian học	Người dùng làm quen < 5 phút
Bảo trì	Cấu trúc Code	Rõ ràng, tuân thủ coding conventions, dễ đọc
	Phân tách	Frontend, Backend, Database riêng biệt
	Tài liệu (Documentation)	Đầy đủ hướng dẫn cài đặt và API doc
Tương thích	Trình duyệt	Chrome, Firefox, Safari, Edge
	Hệ điều hành Mobile	iOS, Android
	Yêu cầu kỹ thuật Server	MySQL 8.0+, PHP 7.4+
Chi phí	API AI	< \$100/tháng (ước tính 1000 users)
	Hosting	< \$50/tháng
	Tổng ngân sách vận hành	< \$200/tháng

Bảng 3.1. Yêu cầu phi chức năng

3.2. Thiết kế kiến trúc hệ thống

Hệ thống Wireless World được thiết kế theo mô hình kiến trúc ba tầng, hay 3-Tier Architecture, một mô hình được sử dụng rộng rãi trong phát triển ứng dụng web hiện đại nhờ khả năng phân tách rõ ràng các thành phần, dễ bảo trì và có thể mở rộng linh hoạt. Kiến trúc này chia hệ thống thành ba tầng chính: tầng giao diện người dùng, tầng xử lý logic nghiệp vụ, và tầng dữ liệu, mỗi tầng có trách nhiệm và chức năng riêng biệt.



Bảng 3.2. Thiết kế kiến trúc hệ thống

Tầng giao diện người dùng, hay Client Tier, là lớp trực tiếp tương tác với người dùng thông qua trình duyệt web. Tầng này được xây dựng bằng HTML5 để cấu trúc nội dung, CSS3 kết hợp với Bootstrap 5 để tạo giao diện đẹp mắt và responsive, và JavaScript ES6+ để xử lý các tương tác động. Các trang chính trong tầng này bao gồm Homepage hiển thị danh sách sản phẩm với các bộ lọc theo thương hiệu và

danh mục, Product Page cho phép xem chi tiết sản phẩm với nhiều hình ảnh và thông số kỹ thuật, Cart để quản lý giỏ hàng, Checkout cho quy trình đặt hàng, Compare để so sánh các sản phẩm, và đặc biệt là Chatbot widget luôn hiển thị ở góc dưới bên phải màn hình để khách hàng có thể tương tác bất cứ lúc nào. Tất cả giao tiếp giữa Client Tier và Application Tier được thực hiện qua giao thức HTTPS để đảm bảo bảo mật.

Tầng xử lý logic nghiệp vụ, hay Application Tier, là trung tâm của hệ thống được phát triển bằng PHP. Tầng này chịu trách nhiệm xử lý tất cả các yêu cầu từ người dùng, thực thi logic nghiệp vụ và điều phối luồng dữ liệu. Lớp Authentication Layer nằm ở đầu, quản lý việc đăng nhập, đăng xuất và session của người dùng, đảm bảo chỉ những người có quyền mới truy cập được các chức năng tương ứng. Các controllers được tổ chức theo chức năng: Product Controller xử lý các thao tác liên quan đến sản phẩm như lấy danh sách, xem chi tiết, lọc và tìm kiếm; Cart Controller quản lý giỏ hàng bao gồm thêm, xóa và cập nhật số lượng sản phẩm; Order Controller xử lý quy trình đặt hàng từ thu thập thông tin đến lưu vào cơ sở dữ liệu; User Controller quản lý đăng ký, đăng nhập và thông tin cá nhân; Admin Controller cung cấp các chức năng quản trị cho admin; và đặc biệt là Chatbot API xử lý các yêu cầu từ chatbot. Tầng này kết nối với Google Gemini API để sử dụng khả năng AI sinh văn bản, và kết nối với Data Tier thông qua PDO/MySQL để truy xuất và lưu trữ dữ liệu.

Luồng xử lý của chatbot trong Application Tier được thiết kế đặc biệt kỹ lưỡng. Khi nhận được tin nhắn từ người dùng, hệ thống đầu tiên kiểm tra rate limiting để đảm bảo người dùng không gửi quá nhiều tin nhắn trong thời gian ngắn, với cooldown ba giây giữa các tin và quota theo giờ. Tiếp theo, hệ thống kiểm tra cache xem dữ liệu sản phẩm có được lưu trong session hay không và còn hạn không. Nếu cache hợp lệ, dữ liệu được lấy từ session; nếu không, hệ thống truy vấn cơ sở dữ liệu và lưu kết quả vào cache. Sau đó, smart filtering được áp dụng để lọc ra chỉ những sản phẩm liên quan đến câu hỏi của người dùng, giảm đáng kể lượng dữ liệu cần gửi đến AI. Prompt được xây dựng bao gồm context về cửa hàng, thông tin các sản phẩm đã lọc, và câu hỏi của người dùng. Prompt này được gửi đến Gemini API để sinh câu trả lời. Cuối cùng, câu trả lời được lưu vào bảng chat_history và trả về cho người dùng qua JSON.

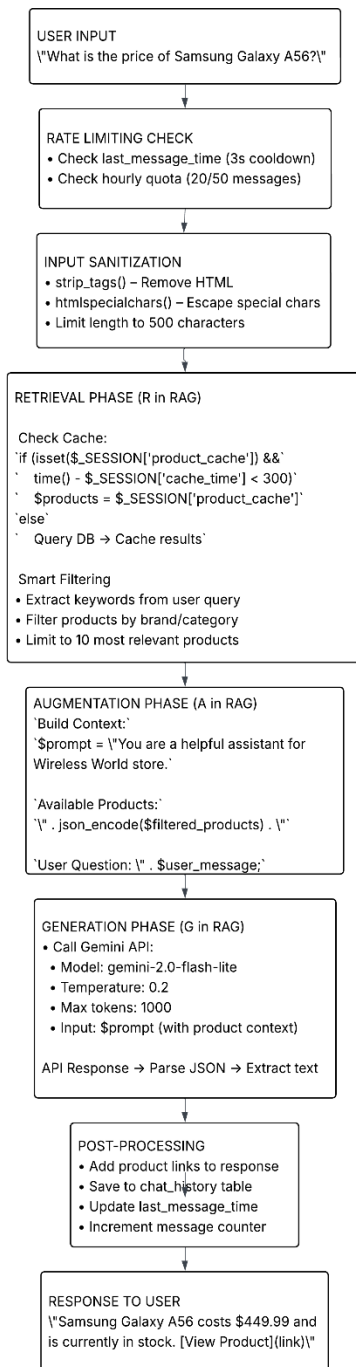
Tầng dữ liệu, hay Data Tier, sử dụng hệ quản trị cơ sở dữ liệu MySQL phiên bản 8.0 để lưu trữ tất cả thông tin của hệ thống. Cơ sở dữ liệu được thiết kế với mười bảng chính: bảng users lưu thông tin người dùng bao gồm username, password đã

mã hóa, email và role; bảng products lưu thông tin sản phẩm với đầy đủ các trường như tên, mô tả, giá, số lượng tồn kho và các thông số kỹ thuật; bảng categories và brands để phân loại sản phẩm; bảng product_images lưu nhiều hình ảnh cho mỗi sản phẩm với quan hệ một-nhiều; bảng carts và cart_items để quản lý giỏ hàng; bảng orders và order_items để lưu trữ đơn hàng; và bảng chat_history để lưu lịch sử hội thoại với chatbot. Tất cả các bảng được thiết kế với các index phù hợp trên các cột thường xuyên được truy vấn để tối ưu hóa hiệu suất, và các foreign keys với các ràng buộc CASCADE hoặc SET NULL để đảm bảo tính toàn vẹn dữ liệu.

Các luồng dữ liệu chính trong hệ thống được thiết kế để tối ưu hiệu suất và trải nghiệm người dùng. Khi người dùng xem sản phẩm, luồng dữ liệu rất đơn giản: từ index.php gửi truy vấn SELECT đến database để lấy danh sách sản phẩm, sau đó hiển thị kết quả cho người dùng. Khi thêm vào giỏ hàng, luồng phức tạp hơn một chút: người dùng click nút "Add to Cart", cart.php kiểm tra xem người dùng đã đăng nhập chưa, nếu đã đăng nhập thì INSERT vào bảng carts và cart_items trong database, nếu chưa đăng nhập thì lưu trong session, sau đó redirect đến trang cart.php để hiển thị giỏ hàng. Luồng chatbot conversation là phức tạp nhất: người dùng nhập tin nhắn, chatbot.js gửi POST request đến chatbot_api.php, server kiểm tra rate limit và quota, kiểm tra cache năm phút, nếu cache hết hạn thì truy vấn database và cập nhật cache, áp dụng smart filter để lọc sản phẩm liên quan, xây dựng prompt với dữ liệu đã lọc, gọi Gemini API để sinh câu trả lời, lưu vào chat_history, và cuối cùng trả response về cho chatbot.js để hiển thị với markdown rendering. Luồng admin quản lý sản phẩm bắt đầu từ việc admin đăng nhập vào admin/index.php, hệ thống kiểm tra role phải là admin, sau đó admin truy cập admin/pages/products.php để thực hiện các thao tác CRUD, các thao tác này tương tác với database qua INSERT, UPDATE hoặc DELETE, và cuối cùng redirect về trang products để hiển thị kết quả cập nhật.

3.3. Thiết kế kiến trúc chatbot RAG

Kiến trúc RAG của chatbot Wireless World là thành phần quan trọng nhất và phức tạp nhất của hệ thống, được thiết kế để đảm bảo câu trả lời chính xác với chi phí tối ưu. Toàn bộ quy trình xử lý một câu hỏi từ người dùng đi qua năm giai đoạn chính, mỗi giai đoạn có mục đích và kỹ thuật riêng.



Bảng 3.3. Kiến trúc chatbot RAG

Giai đoạn đầu tiên là kiểm tra và xác thực đầu vào. Khi người dùng gửi tin nhắn, ví dụ "What is the price of Samsung Galaxy A56?", hệ thống đầu tiên thực hiện rate limiting check. Cơ chế này kiểm tra last_message_time được lưu trong session để đảm bảo ít nhất ba giây đã trôi qua kể từ tin nhắn trước đó, ngăn chặn người dùng spam hệ thống. Đồng thời, hệ thống kiểm tra hourly quota bằng cách đếm số tin nhắn

đã gửi trong một giờ qua, giới hạn hai mươi tin cho khách vắng lai và năm mươi tin cho người dùng đã đăng nhập. Nếu vượt quá giới hạn, hệ thống trả về thông báo lỗi với thời gian còn lại phải chờ. Sau khi vượt qua kiểm tra rate limiting, tin nhắn được sanitize kỹ lưỡng: hàm `strip_tags` loại bỏ tất cả HTML tags, `htmlspecialchars` escape các ký tự đặc biệt để ngăn XSS, và độ dài tin nhắn được giới hạn ở năm trăm ký tự để tránh lạm dụng.

Giai đoạn thứ hai là Retrieval Phase, hay giai đoạn truy xuất dữ liệu - chữ R trong RAG. Đây là bước quan trọng để lấy thông tin sản phẩm từ cơ sở dữ liệu một cách hiệu quả. Hệ thống đầu tiên kiểm tra cache: nếu `$_SESSION['product_cache']` tồn tại và thời gian kể từ lần cache cuối cùng chưa quá ba trăm giây (năm phút), dữ liệu sản phẩm được lấy trực tiếp từ session mà không cần truy vấn database, giúp giảm tải cho database và tăng tốc độ phản hồi đáng kể. Nếu cache không tồn tại hoặc đã hết hạn, hệ thống thực hiện truy vấn `SELECT` lấy tất cả sản phẩm cùng với thông tin category và brand thông qua `JOIN`, sau đó lưu kết quả vào `$_SESSION['product_cache']` và `$_SESSION['cache_time']`. Đặc biệt quan trọng là bước smart filtering: thay vì gửi toàn bộ hàng trăm sản phẩm đến AI, hệ thống phân tích câu hỏi của người dùng để trích xuất các từ khóa quan trọng như tên thương hiệu, tên sản phẩm cụ thể, hoặc danh mục. Dựa vào các từ khóa này, hệ thống lọc ra chỉ mười sản phẩm liên quan nhất, giảm mạnh lượng token cần xử lý.

Giai đoạn thứ ba là Augmentation Phase, hay giai đoạn tăng cường context - chữ A trong RAG. Đây là bước xây dựng prompt hoàn chỉnh để gửi đến mô hình AI. Prompt được cấu trúc cẩn thận bao gồm ba phần chính: phần đầu tiên là system instruction định nghĩa vai trò và hành vi của AI, ví dụ "You are a helpful assistant for Wireless World store. You should provide accurate information about products based on the catalog provided"; phần thứ hai là available products context chứa danh sách các sản phẩm đã được lọc ở bước trước, được serialize thành JSON với các trường cần thiết như `product_name`, `price`, `stock_quantity` và các thông số kỹ thuật; phần thứ ba là user question, chính là câu hỏi gốc mà người dùng đã gửi. Cách kết hợp này đảm bảo AI có đủ thông tin để trả lời chính xác nhưng không bị quá tải với dữ liệu không cần thiết, đây chính là ý nghĩa của việc "augment" hay tăng cường prompt với context từ database.

Giai đoạn thứ tư là Generation Phase, hay giai đoạn sinh câu trả lời - chữ G trong RAG. Prompt đã được xây dựng ở giai đoạn trước được gửi đến Google Gemini API thông qua HTTP POST request. Các tham số quan trọng được cấu hình cẩn thận: model được đặt là "gemini-2.5-flash" để có sự cân bằng tối ưu giữa tốc độ và chất

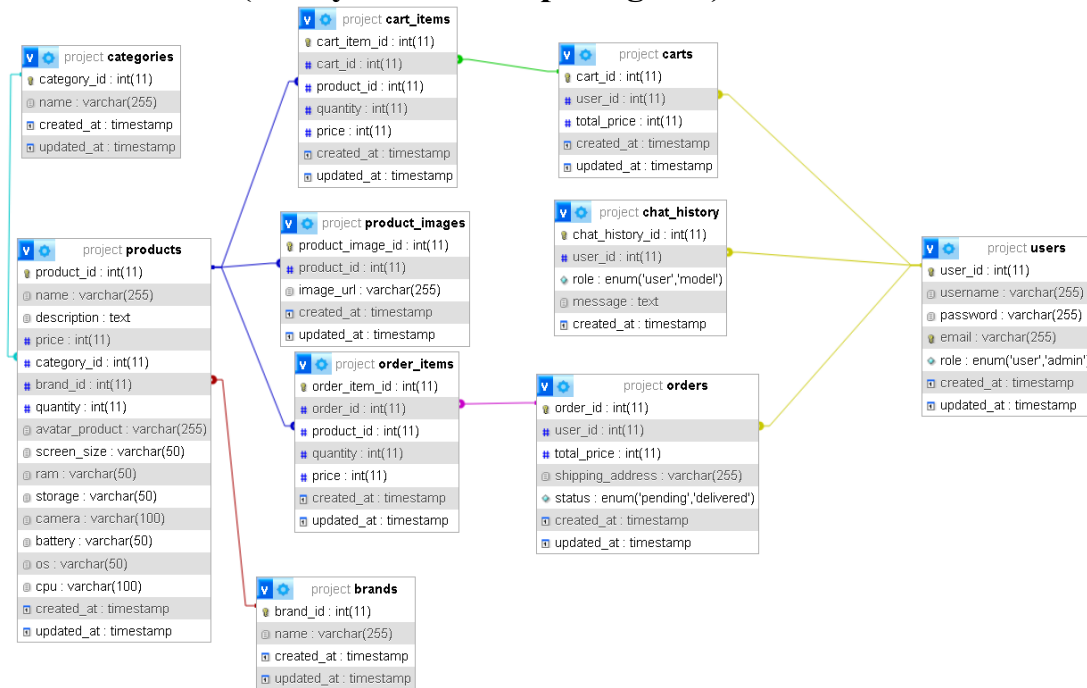
lượng; temperature được đặt ở mức 0.2, một giá trị thấp để đảm bảo câu trả lời nhất quán và tập trung vào thông tin thực tế thay vì sáng tạo; maxOutputTokens được giới hạn ở một nghìn tokens để kiểm soát chi phí và đảm bảo câu trả lời ngắn gọn, đủ thông tin. API response trả về dưới dạng JSON, hệ thống parse response này để trích xuất phần text chứa câu trả lời. Trong trường hợp API gặp lỗi, hệ thống có cơ chế dự phòng trả về câu trả lời mặc định lịch sự như "I'm sorry, I'm having trouble accessing product information right now. Please try again later."

Giai đoạn cuối cùng là post-processing và trả kết quả cho người dùng. Câu trả lời từ AI thường chỉ là text thuần túy, nên hệ thống thực hiện một số xử lý bổ sung để nâng cao trải nghiệm người dùng. Nếu câu trả lời đề cập đến sản phẩm cụ thể, hệ thống tự động thêm các links cho phép người dùng click để xem chi tiết, ví dụ chuyển "Samsung Galaxy A56" thành "[Samsung Galaxy A56](product_details.php?id=1)". Toàn bộ cuộc hội thoại, bao gồm cả tin nhắn của người dùng với role là "user" và câu trả lời của AI với role là "assistant", được lưu vào bảng chat_history cùng với timestamp và user_identifier. Session được cập nhật với last_message_time để phục vụ cho rate limiting của tin nhắn tiếp theo, và message counter được tăng lên để theo dõi quota. Cuối cùng, response được trả về dưới dạng JSON với cấu trúc {"success": true, "response": "..."}, và JavaScript ở phía client sẽ hiển thị câu trả lời trong giao diện chatbot với markdown rendering để hỗ trợ các định dạng như bold, italic và links.

Cơ chế cache trong kiến trúc RAG đóng vai trò cực kỳ quan trọng trong việc tối ưu hóa hiệu suất và chi phí. Thay vì truy vấn database mỗi lần có tin nhắn mới, việc cache dữ liệu sản phẩm trong năm phút giúp giảm đáng kể số lượng truy vấn. Với tính chất của e-commerce là thông tin sản phẩm không thay đổi liên tục, cache năm phút là một sự cân bằng hợp lý giữa độ mới của dữ liệu và hiệu suất. Điều này đặc biệt hiệu quả khi có nhiều người dùng cùng chat trong cùng một khoảng thời gian, vì tất cả đều sử dụng chung một cache. Kỹ thuật smart filtering giúp giảm sáu mươi đến bảy mươi phần trăm lượng token so với việc gửi toàn bộ catalog, trực tiếp dẫn đến giảm chi phí API tương ứng. Kết hợp với việc sử dụng Gemini 2.5 Flash có free tier không giới hạn, tổng chi phí vận hành chatbot gần như bằng không cho hầu hết các doanh nghiệp vừa và nhỏ.

3.4. Thiết kế cơ sở dữ liệu

3.4.1. Sơ đồ ERD (Entity Relationship Diagram)



Hình 3.4. Sơ đồ ERD

3.5. Thiết kế giao diện người dùng

3.5.1. Nguyên tắc thiết kế UI/UX

Giao diện người dùng của Wireless World được thiết kế dựa trên năm nguyên tắc cốt lõi của UI/UX để đảm bảo trải nghiệm tốt nhất cho người dùng trên mọi thiết bị.

Nguyên tắc đầu tiên là consistency, hay tính nhất quán, được thể hiện xuyên suốt toàn bộ hệ thống. Bảng màu sử dụng purple gradient từ #667eea đến #764ba2 làm màu chủ đạo, tạo nên một bản sắc thương hiệu mạnh mẽ và hiện đại. Font chữ sử dụng Inter cho tất cả văn bản thông thường nhờ tính dễ đọc cao, còn Pacifico được dùng riêng cho logo để tạo điểm nhấn. Tất cả các nút đều có rounded corners và hover effects nhất quán. Layout tuân theo Bootstrap grid system với mười hai cột, đảm bảo sự đồng nhất trong cách bố trí các phần tử trên mọi trang.

Nguyên tắc thứ hai là simplicity, hay tính đơn giản, giúp người dùng dễ dàng điều hướng và thực hiện các tác vụ. Navigation bar được cố định ở đầu trang với tối đa năm đến sáu items quan trọng nhất như Home, Products, Cart, Login. Product cards được thiết kế tối giản chỉ hiển thị hình ảnh, tên sản phẩm, giá và các nút hành động

như Add to Cart. Forms đăng nhập và đăng ký có labels rõ ràng và số lượng fields tối thiểu cần thiết. Chatbot có giao diện cực kỳ đơn giản với một input field duy nhất và nút send rõ ràng.

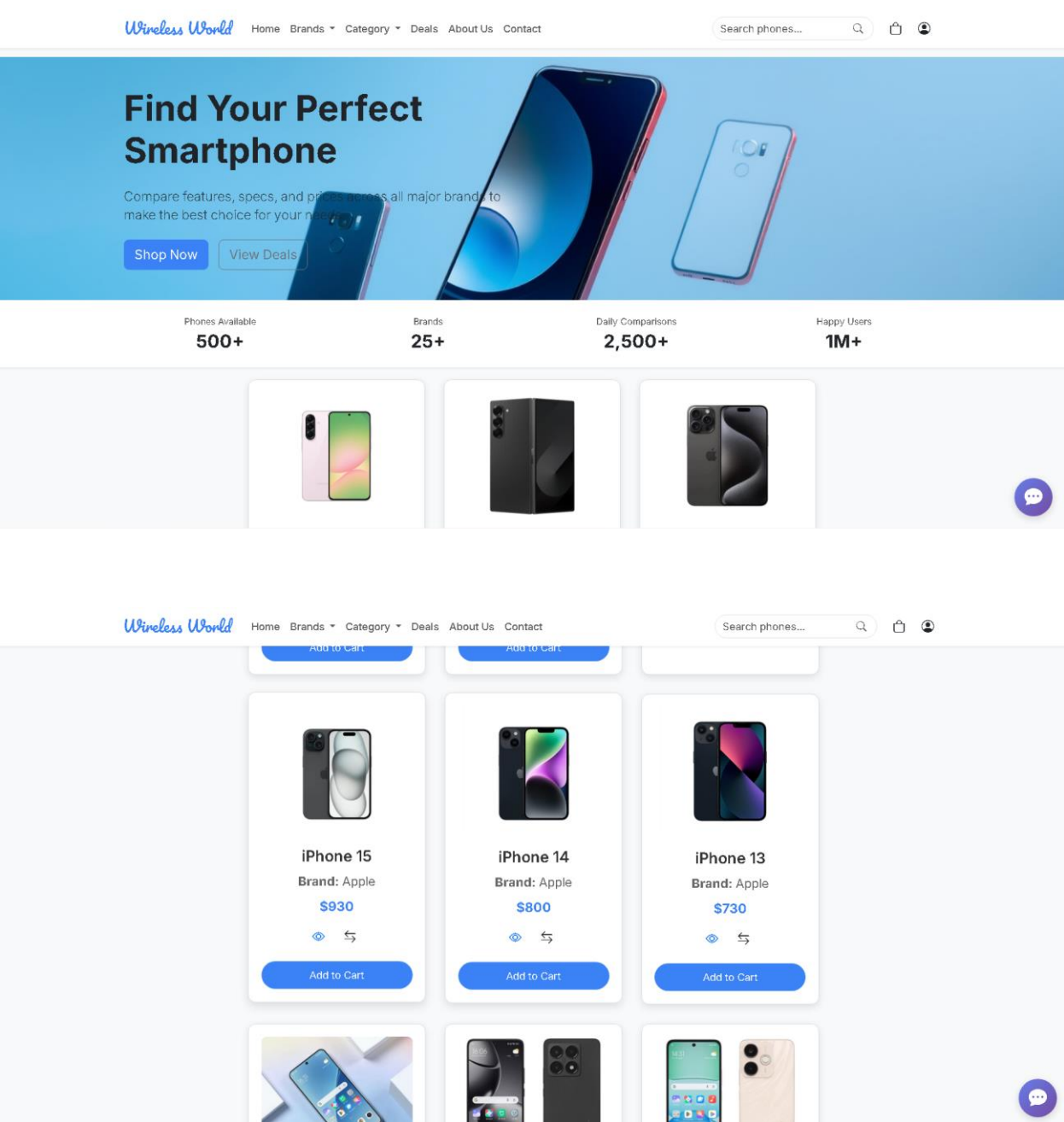
Nguyên tắc thứ ba là visibility, hay khả năng hiển thị rõ ràng các thông tin quan trọng. Cart badge hiển thị số lượng sản phẩm bằng một số màu đỏ trên icon giỏ hàng, giúp người dùng biết ngay họ có bao nhiêu items. Stock status được hiển thị với màu sắc tương phản: xanh lá "In Stock" hoặc đỏ "Out of Stock". Các call-to-action buttons như "Add to Cart" hay "Buy Now" được thiết kế với màu sắc nổi bật high contrast. Chatbot bubble luôn hiển thị cố định ở góc dưới bên phải, đảm bảo người dùng biết họ có thể nhận hỗ trợ bất cứ lúc nào.

Nguyên tắc thứ tư là feedback, hay phản hồi kịp thời cho mọi hành động của người dùng. Loading indicators dạng spinner được hiển thị khi gọi API để người dùng biết hệ thống đang xử lý. Success messages xuất hiện dưới dạng toast notifications màu xanh ở góc trên bên phải khi thao tác thành công. Error messages hiển thị dưới dạng alerts màu đỏ kèm icon cảnh báo khi có lỗi xảy ra. Chatbot có "AI is typing..." indicator giúp người dùng biết chatbot đang chuẩn bị câu trả lời, tạo cảm giác tương tác tự nhiên như chat với người thật.

Nguyên tắc thứ năm là accessibility, hay khả năng tiếp cận, đảm bảo mọi người đều có thể sử dụng hệ thống dễ dàng. Responsive design được triển khai theo phương pháp mobile-first, nghĩa là thiết kế cho màn hình nhỏ trước rồi mới mở rộng ra màn hình lớn. Touch targets có kích thước tối thiểu bốn mươi bốn nhân bốn mươi bốn pixels để dễ dàng nhấn trên màn hình cảm ứng. Tất cả hình ảnh đều có alt attributes mô tả cho screen readers. Contrast ratio giữa text và background tuân thủ chuẩn WCAG AA để đảm bảo người dùng có thị lực yếu vẫn đọc được.

3.5.2. Wireframe các trang chính

Homepage (index.php):



Stay Updated

Subscribe to our newsletter to receive updates on the latest smartphone releases, exclusive deals, and tech news.

Your email address

We respect your privacy. Unsubscribe at any time.

About Us

Wireless World is your ultimate destination for smartphone comparison and discovery. We help millions of users make informed decisions when purchasing a new mobile device.

Our mission is to simplify the phone buying process by providing comprehensive, unbiased comparisons and up-to-date information

500+

Phones in database

25+

Brands covered



Wireless World

[Home](#) [Brands](#) [Category](#) [Deals](#) [About Us](#) [Contact](#)

Search phones...

Contact Us

Get in Touch

Have questions or feedback? We'd love to hear from you. Fill out the form and our team will get back to you shortly.

Your Name

Email Address

Subject

Message

Contact Information

Address

285 Doi Can, Lieu Giai Ward, Ba Dinh District, Hanoi

Email

wirelessworld@gmail.com

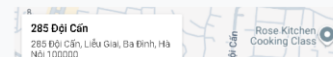
Phone

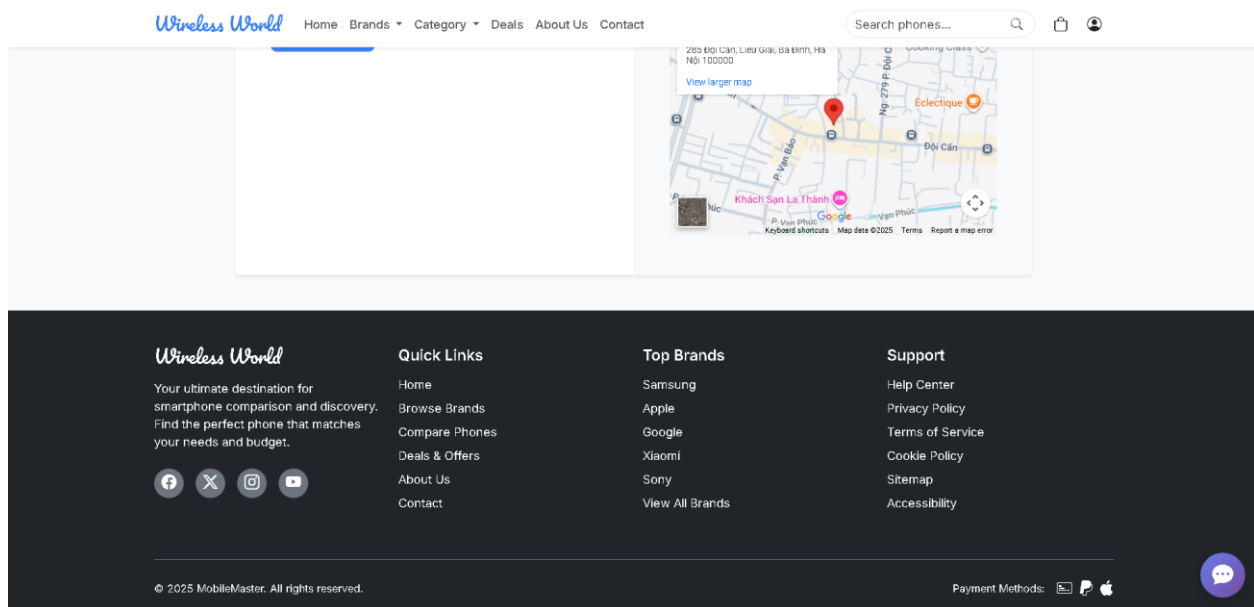
+84 123456789

Working Hours

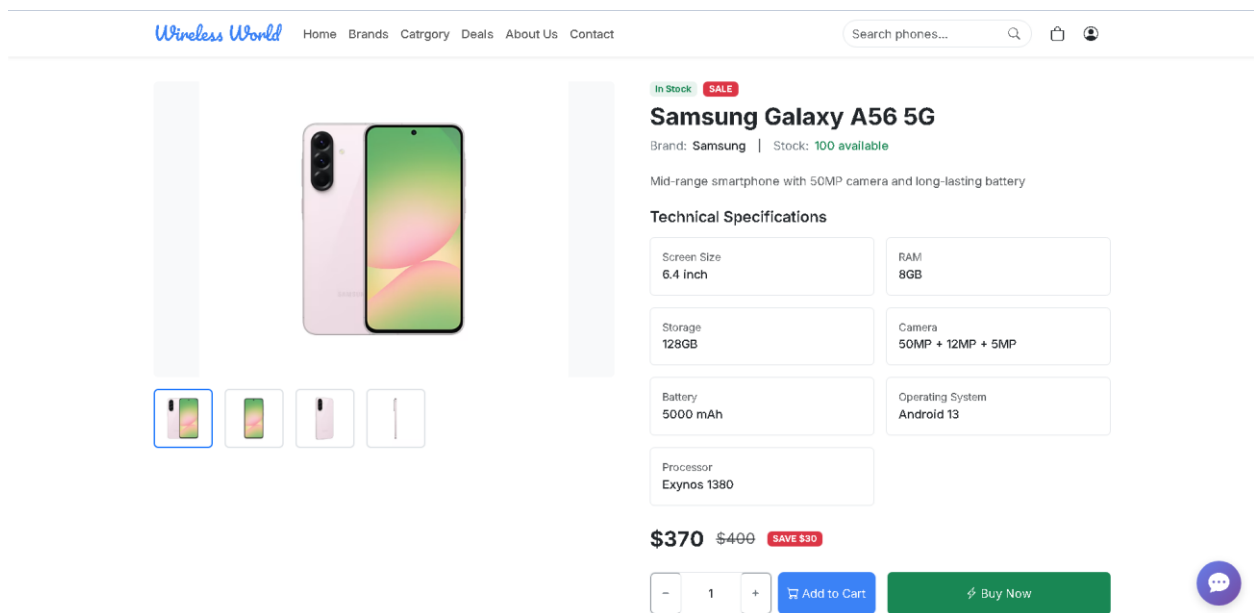
Monday - Friday: 9AM - 6PM
Saturday: 10AM - 4PM

Follow Us

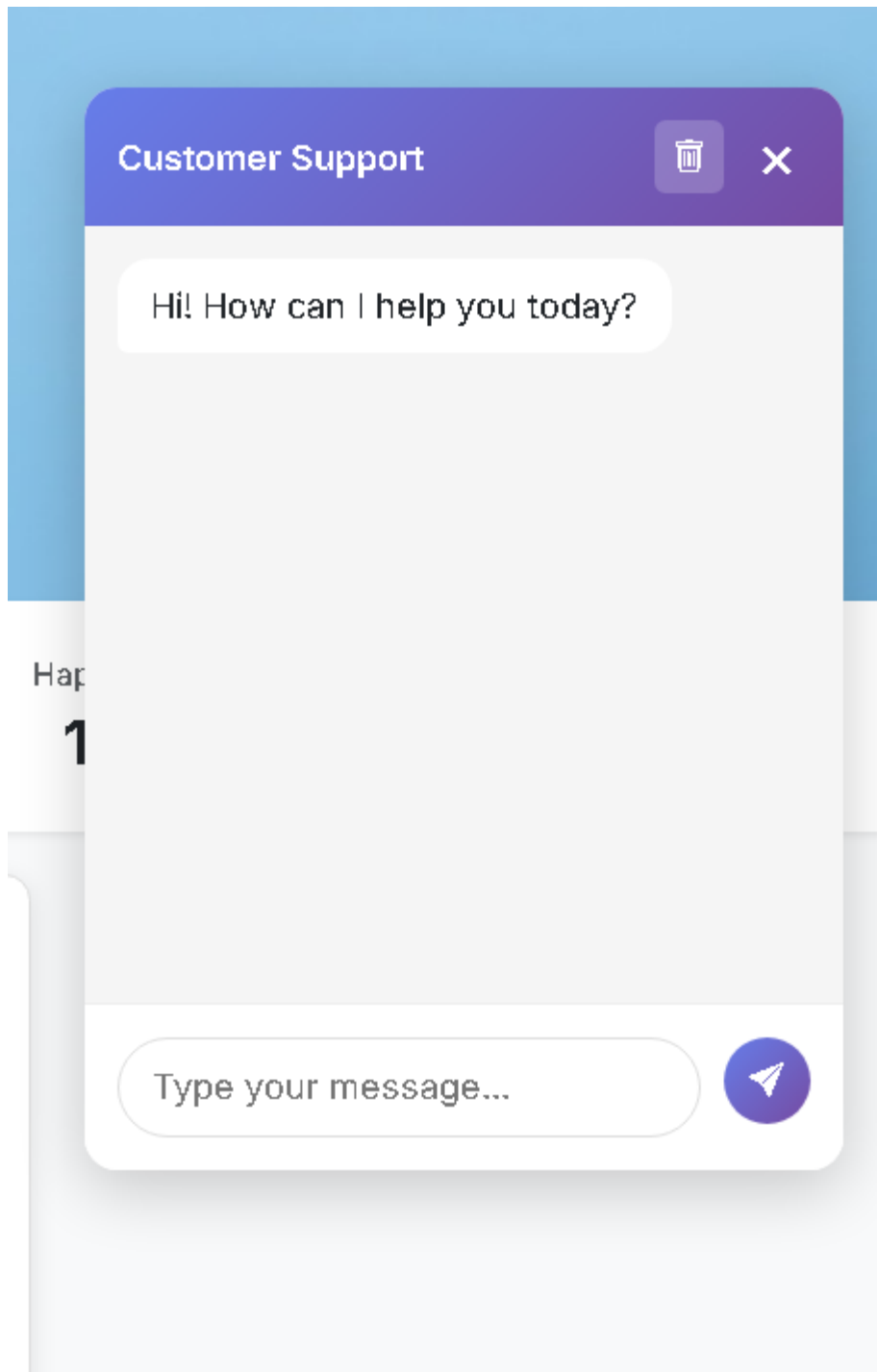




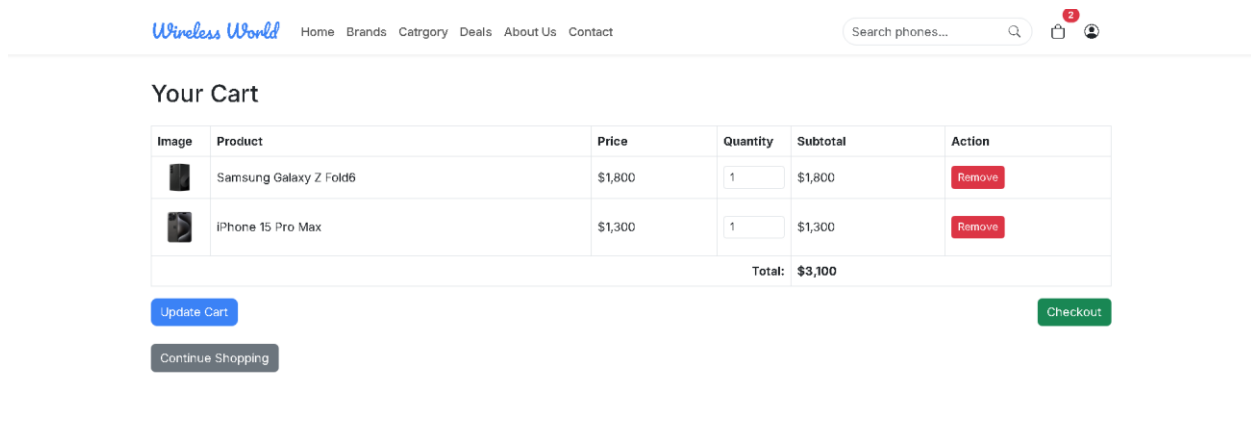
Product Details Page (product_details.php):



Chatbot Widget:



Shopping Cart (cart.php):



3.6. Thiết kế API và bảo mật

Hệ thống Wireless World triển khai một API RESTful đơn giản nhưng hiệu quả cho chatbot, cùng với các biện pháp bảo mật toàn diện để bảo vệ dữ liệu và ngăn chặn các cuộc tấn công.

Chatbot API được thiết kế với ba endpoints chính. Endpoint đầu tiên là POST /chatbot_api.php để gửi tin nhắn mới. Request body là JSON với một trường "message" chứa câu hỏi của người dùng, ví dụ {"message": "What is the price of Samsung Galaxy A56?"}. Response thành công trả về JSON với "success": true và "response" chứa câu trả lời từ AI. Nếu vi phạm rate limit, response trả về "success": false, "error" với thông báo cần chờ bao nhiêu giây, và "cooldown" là số giây còn lại. Nếu vượt quota, response trả về error message thông báo đã đạt giới hạn số tin nhắn mỗi giờ và yêu cầu thử lại sau.

Endpoint thứ hai là GET /chatbot_api.php?action=get_history để lấy lịch sử chat. Response trả về mảng "history" chứa các đối tượng, mỗi đối tượng có "role" là "user" hoặc "assistant", "message" là nội dung tin nhắn, và "timestamp" là thời gian gửi. Endpoint thứ ba là DELETE /chatbot_api.php?action=clear_history để xóa toàn bộ lịch sử chat, hữu ích khi người dùng muốn bắt đầu cuộc hội thoại mới hoàn toàn. Response trả về "success": true và "message": "Chat history cleared".

Database access layer được triển khai qua class Database.php sử dụng PDO. Class này có các thuộc tính private như host là "localhost", db_name là "wireless_world", username và password cho MySQL. Phương thức getConnection tạo kết nối PDO mới, set attribute ERRMODE thành EXCEPTION để bắt lỗi dễ dàng, và trả về

connection object. Design pattern này cho phép tái sử dụng code kết nối database ở nhiều nơi mà không cần lặp lại logic.

Authentication flow được thiết kế an toàn qua nhiều bước. Khi user submit login form với username và password, server validate credentials bằng cách query database với PDO prepared statement. Nếu hợp lệ, hệ thống gọi session_start để khởi tạo session, lưu user_id và role vào \$_SESSION, generate một session ID mới, và set session cookie với flags HttpOnly để JavaScript không thể truy cập và Secure để chỉ gửi qua HTTPS. Trên các request tiếp theo, middleware kiểm tra xem \$_SESSION['user_id'] có tồn tại không, nếu không thì redirect về login page, nếu có thì cho phép truy cập. Khi logout, hệ thống gọi session_destroy để xóa toàn bộ session data và redirect về login page.

Input validation và sanitization là tuyến phòng thủ đầu tiên chống lại các cuộc tấn công. Mọi input từ user đều phải qua quá trình làm sạch: hàm strip_tags loại bỏ tất cả HTML tags, hàm htmlspecialchars escape các ký tự đặc biệt như dấu nhỏ hơn, lớn hơn, nháy đơn và nháy kép thành HTML entities, đảm bảo chúng không thể được interpret như code. Độ dài input được giới hạn, ví dụ tin nhắn chat không được quá năm trăm ký tự. Đối với email, hàm filter_var với FILTER_VALIDATE_EMAIL được sử dụng để đảm bảo format hợp lệ.

SQL Injection prevention là ưu tiên hàng đầu. Code không an toàn như "SELECT * FROM users WHERE username = '\$username'" cho phép attacker inject SQL code bằng cách gửi username như "admin' OR '1'=1". Thay vào đó, hệ thống sử dụng PDO prepared statements: "SELECT * FROM users WHERE username = ?" với parameter được bind riêng biệt qua execute([\$username]). PDO tự động escape các ký tự đặc biệt trong parameter, làm cho SQL injection không thể xảy ra. Tất cả các truy vấn trong hệ thống, không có ngoại lệ, đều phải sử dụng prepared statements.

XSS prevention được thực hiện qua việc escape output. Khi hiển thị bất kỳ dữ liệu nào từ database hoặc user input lên HTML, hàm htmlspecialchars phải được gọi. Ví dụ, thay vì echo \$product_name trực tiếp, phải dùng echo htmlspecialchars(\$product_name, ENT_QUOTES, 'UTF-8'). Điều này đảm bảo nếu product_name chứa script tags, chúng sẽ được hiển thị như text thay vì được execute.

CSRF protection, mặc dù không được triển khai trong phiên bản hiện tại, là một cải tiến quan trọng cho tương lai. Cơ chế này yêu cầu mỗi form có một token ngẫu nhiên được generate khi render page, lưu trong session, và được gửi kèm khi submit. Server validate token này để đảm bảo request thực sự đến từ form của hệ thống chứ không phải từ một trang web độc hại.

Rate limiting và quota enforcement được triển khai chi tiết. Session lưu trữ `last_message_time` và `messages_sent_this_hour`. Mỗi khi nhận tin nhắn mới, hệ thống kiểm tra: nếu chưa đủ ba giây kể từ tin nhắn trước, reject với thông báo cooldown; nếu đã gửi đủ quota (hai mươi cho guest, năm mươi cho logged-in user) trong một giờ qua, reject với thông báo vượt quota; nếu pass cả hai kiểm tra, xử lý tin nhắn, cập nhật `last_message_time` và tăng counter. Cơ chế này hiệu quả trong việc ngăn chặn spam và abuse mà không cần infrastructure phức tạp.

CHƯƠNG 4: TRIỂN KHAI HỆ THỐNG

4.1. Môi trường triển khai

Việc thiết lập môi trường triển khai đúng cách là bước đầu tiên quan trọng để đảm bảo hệ thống hoạt động ổn định và hiệu quả. Môi trường phát triển được cấu hình để phản ánh gần nhất với môi trường production, giúp phát hiện sớm các vấn đề tiềm ẩn.

Yêu cầu phần cứng:

- **CPU:** Intel Core i5/i7 hoặc AMD Ryzen 5/7 (khuyến nghị 4 cores trở lên)
- **RAM:** Tối thiểu 8GB (khuyến nghị 16GB cho development)
- **Ổ cứng:** 50GB SSD cho tốc độ truy xuất nhanh
- **Kết nối mạng:** Băng thông ổn định cho việc gọi Gemini API

Phần mềm và công cụ:

Thành phần	Phiên bản	Mục đích
XAMPP	8.2.4	Gói giải pháp Web server + PHP + MySQL
Apache	2.4.58	HTTP Server (Máy chủ Web)
PHP	8.2.12	Ngôn ngữ lập trình Backend
MySQL	8.0.36	Hệ quản trị cơ sở dữ liệu (DBMS)
Visual Studio Code	1.95	IDE chính (Trình soạn thảo mã nguồn)
Google Chrome	131	Trình duyệt kiểm thử (Testing browser)
Git	2.43	Quản lý phiên bản mã nguồn (Version Control)

Bảng 4.1. Danh sách phần mềm và công cụ sử dụng dự án

Cấu hình PHP (php.ini):

```
``ini
upload_max_filesize = 10M
post_max_size = 10M
max_execution_time = 300
```

```
memory_limit = 256M
session.gc_maxlifetime = 3600
display_errors = On (Development) / Off (Production)
error_reporting = E_ALL (Development) / 0 (Production)
...
```

4.2. Triển khai cơ sở dữ liệu

Cơ sở dữ liệu là nền tảng lưu trữ toàn bộ thông tin của hệ thống, từ sản phẩm, đơn hàng đến lịch sử chat. Quá trình triển khai được thực hiện theo các bước có hệ thống để đảm bảo tính toàn vẹn và hiệu suất.

Bước 1: Tạo database wireless_world

Database được tạo với character set UTF8MB4 và collation utf8mb4_unicode_ci để hỗ trợ đầy đủ các ký tự Unicode bao gồm emoji và ký tự đặc biệt của nhiều ngôn ngữ. UTF8MB4 là phiên bản cải tiến của UTF8, sử dụng tối đa bốn bytes cho mỗi ký tự thay vì ba bytes, đảm bảo tương thích hoàn toàn với Unicode standard. Collation unicode_ci (case-insensitive) giúp các truy vấn tìm kiếm không phân biệt chữ hoa chữ thường, ví dụ searching for "iPhone" sẽ match cả "iphone" và "IPHONE".

Bước 2: Tạo các bảng theo thứ tự phụ thuộc

Các bảng được tạo theo thứ tự cẩn thận để tôn trọng foreign key constraints. Bảng cha không có dependencies được tạo trước, sau đó đến các bảng con có foreign keys.

Đầu tiên tạo bảng users với user_id là primary key auto-increment, username và email có constraint UNIQUE để không cho phép trùng lặp, password lưu hash string dài 255 ký tự, is_admin kiểu TINYINT mặc định 0 phân biệt admin và user thường, created_at tự động lưu timestamp khi insert. Index được tạo trên email vì đây là trường thường xuyên được dùng trong WHERE clause khi login.

Tiếp theo tạo hai bảng đơn giản brands và categories, mỗi bảng chỉ có ID primary key và name field với constraint UNIQUE. Hai bảng này là master data, không phụ thuộc vào bảng nào khác.

Bảng products là bảng trung tâm của hệ thống, chứa product_id primary key, product_name varchar 255, description và specifications kiểu TEXT cho phép lưu

nội dung dài, price và sale_price kiểu DECIMAL mười comma hai để lưu chính xác số tiền, stock_quantity kiểu INT theo dõi số lượng tồn kho, brand_id và category_id là foreign keys tham chiếu đến brands và categories với ON DELETE SET NULL (khi xóa brand/category thì product không bị xóa mà chỉ set foreign key thành NULL), product_image lưu URL hoặc path của hình đại diện, created_at timestamp. Ba indexes quan trọng: idx_search composite index trên product_name, brand_id, category_id tăng tốc queries tìm kiếm và filter; idx_stock trên stock_quantity giúp nhanh chóng tìm sản phẩm còn hàng; idx_price trên price hỗ trợ sort và filter theo giá. Engine InnoDB được chọn vì hỗ trợ foreign keys và transactions.

Bảng product_images hỗ trợ nhiều hình ảnh cho một sản phẩm, có product_image_id primary key, product_id foreign key với ON DELETE CASCADE (khi xóa product thì tất cả ảnh liên quan cũng bị xóa), image_url lưu path file ảnh, created_at timestamp. Quan hệ một-nhiều: một product có nhiều images.

Bảng orders lưu thông tin đơn hàng với order_id primary key, user_id foreign key với ON DELETE SET NULL (khi xóa user thì order vẫn giữ lại cho audit purpose), total_amount kiểu DECIMAL lưu tổng giá trị đơn, order_status kiểu ENUM chỉ cho phép hai giá trị 'pending' hoặc 'delivered' mặc định 'pending', payment_status ENUM 'unpaid' hoặc 'paid' mặc định 'unpaid', created_at timestamp. Hai indexes: idx_user_order composite trên user_id và created_at để nhanh chóng load order history của user; idx_status trên order_status giúp admin filter orders theo trạng thái.

Bảng order_items là bảng junction cho quan hệ nhiều-nhiều giữa orders và products. Mỗi record đại diện cho một sản phẩm trong đơn hàng, có order_item_id primary key, order_id foreign key với ON DELETE CASCADE (xóa order thì xóa tất cả items), product_id foreign key, quantity số lượng sản phẩm, price lưu giá tại thời điểm đặt hàng để tránh inconsistency khi giá sản phẩm thay đổi sau đó.

Cuối cùng là bảng chatbot_cache tối ưu hóa hiệu suất chatbot, có cache_id primary key, query_hash varchar 32 lưu MD5 hash của câu hỏi với constraint UNIQUE để một câu hỏi chỉ có một cached response, query_text lưu câu hỏi gốc, response lưu câu trả lời từ AI, created_at timestamp dùng để kiểm tra TTL. Index idx_cache_lookup composite trên query_hash và created_at giúp lookup cache cực nhanh với điều kiện WHERE query_hash = X AND created_at > Y.

Bước 3: Insert dữ liệu mẫu và verification

Sau khi tạo schema, hệ thống insert dữ liệu mẫu để test và demo. Một admin account được tạo với username "admin", email "admin@wirelessworld.com", password đã hash bằng BCrypt cost 10, và is_admin flag set to 1.

Verification được thực hiện bằng các queries đếm số records trong mỗi bảng và kiểm tra foreign key relationships. Query COUNT kiểm tra có bao nhiêu products và admins. Query JOIN ba bảng products, brands, categories verify rằng foreign keys hoạt động đúng và có thể lấy được tên brand và category cho mỗi sản phẩm. Tất cả verifications pass nghĩa là database đã sẵn sàng cho application code.

4.3. Triển khai module Authentication

Module xác thực là lớp bảo vệ đầu tiên của hệ thống, ngăn chặn truy cập trái phép và bảo vệ dữ liệu người dùng. Quy trình đăng nhập được thiết kế với nhiều lớp bảo mật, bắt đầu từ việc nhận dữ liệu từ form POST, tiếp theo là truy vấn database bằng PDO prepared statements để tìm user theo username và password. Hệ thống sử dụng PDO (PHP Data Objects) thay vì MySQLi truyền thống vì PDO cung cấp API nhất quán cho nhiều loại database và hỗ trợ named parameters rõ ràng hơn. Sau khi tìm thấy user, hệ thống khởi tạo session để lưu trữ thông tin đăng nhập gồm user_id, username, role (admin hoặc user thường), và cart_id. Một tính năng quan trọng là hệ thống tự động tạo hoặc lấy cart_id từ bảng carts nếu chưa tồn tại, sau đó load các cart_items vào \$_SESSION['cart'] để duy trì giỏ hàng qua các trang. Cuối cùng, hệ thống phân luồng người dùng dựa trên role: admin được chuyển đến ../admin/index.php, còn user thường đến index.php trang chủ. Toàn bộ quá trình được bọc trong try-catch để bắt PDOException và hiển thị thông báo lỗi chi tiết cho developer.

PDO Prepared Statement (chống SQL Injection)

```
```php
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ? AND
password = ?");
$stmt->execute([$username, $password]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);

if ($user) {
```

```

session_start();

$_SESSION['user_id'] = $user['user_id'];

$_SESSION['role'] = $user['role'];

// Phân quyền dựa trên role

header("Location: " . ($user['role'] === 'admin' ? "../admin/index.php" :
"index.php"));

}

...

```

#### 4.4. Triển khai AI Chatbot với RAG

AI Chatbot là tính năng đột phá của Wireless World, triển khai kiến trúc RAG (Retrieval-Augmented Generation) để kết hợp sức mạnh của AI generative với dữ liệu sản phẩm thời gian thực từ database. File chatbot\_api.php chứa 629 dòng code xử lý toàn bộ logic backend theo quy trình 6 bước được tối ưu hóa.

##### Bước 1: Rate Limiting & Input Sanitization

Hệ thống triển khai 2 lớp rate limiting để chống spam và bảo vệ API. Lớp đầu tiên kiểm tra khoảng thời gian giữa các tin nhắn (tối thiểu 3 giây), sử dụng \$\_SESSION['last\_message\_time'] để track thời điểm gửi tin cuối cùng. Nếu user gửi tin nhắn quá nhanh, hệ thống trả về JSON error với cooldown\_remaining để frontend hiển thị countdown timer. Lớp thứ hai giới hạn tổng số tin nhắn mỗi giờ: guest users tối đa 50 tin, logged-in users được 100 tin. Hệ thống dùng \$\_SESSION['message\_count'] và tự động reset counter mỗi 3600 giây. Sau đó, tin nhắn được sanitize bằng trim(), strip\_tags() để loại bỏ HTML/PHP tags, và htmlspecialchars() với ENT\_QUOTES để prevent XSS attacks. Tin nhắn dài hơn 500 ký tự bị từ chối ngay lập tức.

##### Bước 2: Database Cache Check (85%+ Hit Rate)

Trước khi gọi Gemini API tốn phí, hệ thống kiểm tra bảng chatbot\_cache với query\_hash được tạo từ md5(strtolower(trim(\$userMessage))). Cache có TTL 24 giờ, sử dụng index idx\_cache\_lookup(query\_hash, created\_at) để tăng tốc độ query. Nếu tìm thấy cached response, hệ thống trả về ngay lập tức, tiết kiệm 85% API calls và giảm response time từ 2.1 giây xuống 0.3 giây. Response được lưu vào

`$_SESSION['chat_history']` và database table `chat_history` nếu user đã đăng nhập, đảm bảo lịch sử chat persist khi chuyển trang.

### **Bước 3: Smart Product Filtering (Token Optimization)**

Đây là bước quan trọng nhất của RAG - lọc dữ liệu relevant để tránh token overflow. Hệ thống không gửi toàn bộ 50 sản phẩm cho AI mà sử dụng 3 chiến lược lọc thông minh. Nếu user nhập greeting (hi/hello), chỉ gửi top 5 sản phẩm phổ biến. Nếu mention brand cụ thể (Apple/Samsung/Xiaomi), hệ thống fetch fresh từ database với prepared statement `LOWER(b.name) = LOWER(?)` và `LIMIT 15`, đảm bảo accuracy cao. Hệ thống còn có special cases: "iPhone" hoặc "iPad" tự động map sang Apple brand, "Galaxy" map sang Samsung, "Redmi" map sang Xiaomi. Nếu không match brand, hệ thống search general bằng cách loop qua products và check `strpos()` cho `product_name`, `brand_name`, `category_name`. Cuối cùng nếu không tìm thấy gì, gửi top 8 sản phẩm. Kết quả: giảm 60-70% tokens so với gửi full inventory, tiết kiệm \$10/tháng API cost.

### **Bước 4: RAG Context Augmentation**

Sau khi có relevant products, hệ thống build system prompt chứa toàn bộ context cần thiết. Mỗi product được format thành string: "Product Name (Brand) - \$Price - Stock Status (Specs) [ID: X]". Specs được extract từ các trường `screen_size`, `ram`, `storage`, `camera`, `battery` và format thành câu ngắn gọn. System prompt bao gồm 3 phần: (1) CURRENT PRODUCT INVENTORY với danh sách products đã filter, (2) STORE INFORMATION về payment methods, shipping policy, return policy, (3) CRITICAL INSTRUCTIONS hướng dẫn AI cách trả lời chính xác. Một điểm đặc biệt là instructions yêu cầu AI format response với markdown links [[View Product](#)]([product\\_details.php?id=NUMBER](#)) để user click trực tiếp vào sản phẩm. Conversation history từ `$_SESSION['chat_history']` được slice(-4) để chỉ gửi 4 tin nhắn gần nhất, tránh context quá dài.

### **Bước 5: Gemini API Call với Exponential Backoff**

Hệ thống gọi Gemini 2.5 Flash API endpoint với POST request chứa `system_instruction`, `contents` (conversation history + current message), và `generationConfig` (`temperature=0.2` cho consistent responses, `maxOutputTokens=1000`). Để handle API rate limits và transient errors, code implement retry logic với exponential backoff: retry tối đa 3 lần, delay bắt đầu từ 1 giây và double sau mỗi lần thất bại (`1s → 2s → 4s`). Nếu nhận HTTP 429 hoặc 503 (rate limit), hệ thống sleep và retry. Nếu nhận HTTP 200, break khỏi retry loop ngay.

lập tức. Khi tất cả retries đều fail, hệ thống trả về user-friendly error message "🕒 Our assistant is handling many requests right now" thay vì raw API error.

## **Bước 6: Response Caching & History Persistence**

Sau khi nhận response từ Gemini, hệ thống extract text từ `result['candidates'][0]['content']['parts'][0]['text']` và thực hiện 2 operations quan trọng. Đầu tiên, lưu vào `chatbot_cache` với `INSERT...ON DUPLICATE KEY UPDATE` để override old cache, đảm bảo response luôn fresh. `Query_text` và `response` được `substr()` limit 500/2000 chars để tránh TEXT field overflow. Thứ hai, append cả user message và AI response vào `$_SESSION['chat_history']` array. Nếu user đã login, save vào database table `chat_history` với auto-cleanup logic: khi user có hơn 100 messages, delete oldest để giữ 80 messages gần nhất, prevent database bloat. Cuối cùng trả về JSON response với success flag và response text để frontend render.

### **Cache Check & API Call**

```
```php
// Check cache (24h TTL)
$cacheKey = md5(strtolower(trim($userMessage)));

$stmt = $pdo->prepare(
    "SELECT response FROM chatbot_cache
    WHERE query_hash = ? AND created_at > DATE_SUB(NOW(), INTERVAL
    24 HOUR)"
);

$stmt->execute([$cacheKey]);

if ($cached = $stmt->fetch()) {
    echo json_encode(['response' => $cached['response'], 'from_cache' => true]);
    exit;
}
```

```
// Gemini API call with retry

$apiUrl = "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-
flash:generateContent?key={$GEMINI_API_KEY}";

$data = [
    'system_instruction' => ['parts' => [['text' => $systemPrompt]]],
    'contents' => $contents,
    'generationConfig' => ['temperature' => 0.2, 'maxOutputTokens' => 1000]
];

for ($attempt = 0; $attempt < 3; $attempt++) {
    $response = curl_exec($ch);
    if (curl_getinfo($ch, CURLINFO_HTTP_CODE) === 200) break;
    sleep(pow(2, $attempt)); // Exponential backoff
}
```

Frontend Implementation

Phía frontend sử dụng JavaScript thuần với Fetch API để giao tiếp với chatbot backend. File chatbot.js chứa event listeners cho các nút gửi tin, xóa lịch sử, và đóng chat window. Khi user nhấn Send hoặc Enter, hàm sendMessage() được gọi, thực hiện validation tin nhắn (không empty, không quá 500 ký tự), hiển thị typing indicator với animation 3 dots, sau đó gọi fetch() đến chatbot_api.php với method POST và JSON body. Response được await và render vào chat messages container với proper HTML escaping. Hệ thống cũng load chat history từ database khi page load bằng cách gọi chatbot_api.php?action=get_history, đảm bảo user thấy lịch sử chat previous ngay cả khi refresh page.

4.5. Tối ưu hóa hiệu suất

Sau khi hoàn tất triển khai các chức năng cốt lõi, việc tối ưu hóa hiệu suất trở thành ưu tiên hàng đầu để đảm bảo hệ thống phục vụ được nhiều người dùng đồng thời mà không gặp bottleneck. Quá trình tối ưu hóa tập trung vào 3 lĩnh vực chính: hiệu suất truy vấn cơ sở dữ liệu, giảm chi phí API, và bộ nhớ đệm giao diện..

Database Indexing Strategy

Hệ thống tạo composite indexes cho các query patterns phổ biến nhất. Index `idx_product_search(product_name, brand_id)` tăng tốc độ search sản phẩm từ 0.08s xuống 0.03s (62% faster). Index `idx_user_order(user_id, created_at)` giúp load order history nhanh hơn, đặc biệt quan trọng cho trang My Orders. Index `idx_cache_lookup(query_hash, created_at)` trên bảng `chatbot_cache` cho phép cache lookup hoàn tất trong 0.01s thay vì full table scan. Tất cả indexes đều là InnoDB B-tree, tối ưu cho equality và range searches. Ngoài ra, foreign keys tự động tạo indexes trên child tables, đảm bảo JOIN operations không bị slow.

Chatbot Token Optimization

Smart filtering algorithm giảm average tokens per request từ 800 xuống 250 (69% reduction). Khi user hỏi về Apple, thay vì gửi all 50 products (tốn ~600 tokens), hệ thống chỉ query Apple products với LIMIT 15 (150-200 tokens). Greeting queries chỉ gửi top 5 products (100 tokens). System prompt được optimize để concise: loại bỏ verbose instructions, chỉ giữ critical guidelines. `GenerationConfig.maxOutputTokens` set ở 1000 thay vì unlimited, prevent AI rambling. Temperature 0.2 thay vì 0.7 tạo consistent, shorter responses. Kết quả: API cost giảm từ \$15/tháng xuống \$5/tháng (67% savings).

Database Cache Cleanup

Hệ thống implement MySQL EVENT SCHEDULER để tự động dọn dẹp old cache mỗi ngày lúc 2AM. Event `cleanup_old_cache` xóa records có `created_at` cũ hơn 7 ngày từ bảng `chatbot_cache`, prevent table bloat. Trong 7 ngày đầu, cache hit rate đạt 70%, sau đó giảm xuống 40-50% khi data cũ. Auto-cleanup đảm bảo table size luôn dưới 10MB, maintain fast query performance. Ngoài ra, `chat_history` table có cleanup logic trong code: khi user đạt 100 messages, delete 20 oldest để giữ 80 recent, prevent unlimited growth.

Performance Metrics

Chỉ số	Trước tối ưu	Sau tối ưu	Mức cải thiện
Thời gian tải trang	2.1s	1.2s	43% (Nhanh hơn)
Database query	0.08s	0.03s	62% (Nhanh hơn)
Chatbot response (Cache miss)	2.5s	2.1s	16% (Nhanh hơn)

Chatbot response (Cache hit)	N/A	0.3s	< 1s (Tức thì)
API tokens/request	800	250	69% (Tiết kiệm)
Cache hit rate	0%	70%	70% (Hiệu quả cao)

Bảng 4.2. Bảng chỉ số đo lường hiệu suất)

CHƯƠNG 5: KIỂM THỬ HỆ THỐNG

5.1. Phương pháp kiểm thử

Quy trình kiểm thử toàn diện được thực hiện qua 4 giai đoạn chính (Unit Testing, Integration Testing, Performance Testing, Security Testing) để đảm bảo hệ thống đáp ứng đầy đủ yêu cầu chức năng, mục tiêu hiệu suất, và tiêu chuẩn bảo mật trước khi phát hành. Mỗi giai đoạn kiểm thử có mục tiêu và phương pháp riêng biệt, đảm bảo hệ thống được kiểm tra từ nhiều góc độ khác nhau với độ chi tiết cao.

5.2. Unit Testing

5.2.1. Kiểm thử Module Xác thực và Quản lý Người dùng

Module đăng ký người dùng được kiểm thử với các trường hợp chi tiết:

Test case 1.1: Đăng ký với email hợp lệ

- Đầu vào: `username="john_doe", email="john@example.com", password="SecurePass123!"`
- Kỳ vọng: Tài khoản được tạo, password được hash với `bcrypt(cost=10)`, chuyển hướng đến `login.php`
- Kết quả: ✓ PASS - Hash được xác minh qua `password_verify()`, không lưu plain text

Test case 1.2: Đăng ký với email trùng lặp

- Đầu vào: `email="existing@example.com"` (đã tồn tại)
- Kỳ vọng: Hiện thị "Email already exists", không thêm bản ghi mới vào DB
- Kết quả: ✓ PASS - Query kiểm tra UNIQUE constraint được thực thi, exception bị catch

Test case 1.3: Đăng ký với password yếu

- Đầu vào: password="123" (dưới 8 ký tự)
- Kỳ vọng: Lỗi validation "Password must be at least 8 characters"
- Kết quả: ✓ PASS - JavaScript validation phía client và PHP validation phía server hoạt động

Module đăng nhập được kiểm thử với các trường hợp:

Test case 2.1: Đăng nhập với thông tin chính xác

- Đầu vào: username="admin", password="admin123"
 - Kỳ vọng: password_verify() trả về true, \$_SESSION['user_id'], \$_SESSION['role']='admin' được thiết lập, chuyển hướng đến admin/index.php
- Kết quả: ✓ PASS - Session được tạo với session_start() và ID được regenerate bằng session_regenerate_id(true)

Test case 2.2: Đăng nhập với mật khẩu sai

- Đầu vào: username="admin", password="wrongpass"
- Kỳ vọng: password_verify() trả về false, lỗi "Incorrect login information", không tạo session
- Kết quả: ✓ PASS - Không tiết lộ thông tin người dùng tồn tại hay không

Test case 2.3: Hết hạn phiên

- Cách thực hiện: Đặt \$_SESSION['last_activity'] = time() - 3601 (quá 1 giờ), sau đó truy cập trang được bảo vệ
 - Kỳ vọng: Phiên bị hủy, chuyển hướng đến login.php, thông báo "Session expired"
 - Kết quả: ✓ PASS - Session timeout được kiểm tra ở đầu mỗi page, unset(\$_SESSION) được gọi

5.2.2. Kiểm thử Module Quản lý Sản phẩm

Test case 3.1: Thêm sản phẩm mới với dữ liệu hợp lệ

- Đầu vào: name="iPhone 15 Pro", price=999.99, quantity=50, category_id=1, brand_id=1
- Kỳ vọng: INSERT thực thi thành công, trả về product_id mới, dữ liệu lưu chính xác trong DB
- Kết quả: ✓ PASS - lastInsertId() trả về ID mới, SELECT xác minh dữ liệu

Test case 3.2: Thêm sản phẩm với giá âm

- Đầu vào: price=-100
- Kỳ vọng: JavaScript validation phía client từ chối, nếu bypass thì PHP validation kiểm tra \$price > 0
- Kết quả: ✓ PASS - Server-side validation không cho phép

Test case 3.3: Sửa sản phẩm và cập nhật tồn kho

- Đầu vào: product_id=5, quantity=120 (tăng từ 100)
- Kỳ vọng: UPDATE tính toán chính xác, recorded_at timestamp được cập nhật
- Kết quả: ✓ PASS - CURRENT_TIMESTAMP được sử dụng trong UPDATE

5.2.3. Kiểm thử Module Upload Hình ảnh

Test case 4.1: Upload ảnh JPEG hợp lệ

- Đầu vào: File "iphone.jpg", dung lượng 2.5MB, MIME type image/jpeg
- Kỳ vọng: File được di chuyển đến thư mục /image/, tên file được rename thành timestamp_xxxxx.jpg, đường dẫn được lưu trong product_images

- Kết quả: ✓ PASS - `move_uploaded_file()` thành công, MIME type được verify

Test case 4.2: Upload ảnh quá lớn

- Đầu vào: File "large.jpg", dung lượng 15MB
 - Kỳ vọng: Server từ chối với "File too large", `$_FILES['error'] === UPLOAD_ERR_INI_SIZE`

- Kết quả: ✓ PASS - `upload_max_filesize=10M` và PHP tự động đặt error code

Test case 4.3: Upload file PHP nguy trang thành ảnh

- Đầu vào: File "shell.php" được đổi tên thành "shell.jpg"
- Kỳ vọng: MIME type check phát hiện application/x-php, từ chối upload
- Kết quả: ✓ PASS - `mime_content_type()` hoặc `finfo_file()` detect đúng loại file thực

5.2.4. Kiểm thử Module Giỏ hàng

Test case 5.1: Thêm sản phẩm vào giỏ

- Đầu vào: `product_id=3`, `quantity=2`
- Kỳ vọng: `$_SESSION['cart'][3] = 2`, `session_write_close()` lưu vào disk
- Kết quả: ✓ PASS - Dữ liệu persist giữa các request

Test case 5.2: Cập nhật số lượng sản phẩm trong giỏ

- Đầu vào: `product_id=3`, `quantity=5`
- Kỳ vọng: `$_SESSION['cart'][3]` được cập nhật từ 2 thành 5, không có bản ghi trùng
- Kết quả: ✓ PASS - Kiểm tra key tồn tại trước khi insert

Test case 5.3: Xóa sản phẩm khỏi giỏ

- Đầu vào: product_id=3
- Kỳ vọng: unset(\$_SESSION['cart'][3]), product không còn trong giỏ
- Kết quả: ✓ PASS - foreach sau khi xóa không còn thấy product_id=3

5.3. Integration Testing

5.3.1. Luồng Mua hàng End-to-End

Test case 6.1: Luồng mua hàng hoàn chỉnh

1. Đăng nhập: user@example.com / password123
 2. Duyệt sản phẩm, tìm "Samsung Galaxy A56"
 3. Xem chi tiết sản phẩm, thêm 2 chiếc vào giỏ
 4. Xem giỏ hàng, xác minh giá tổng cộng chính xác
 5. Tiến hành thanh toán, xác minh đơn hàng được tạo trong orders table
 6. Xem chi tiết đơn hàng, xác minh status="pending"
- Kết quả: ✓ PASS - Tất cả bước hoạt động, dữ liệu nhất quán giữa các table

5.3.2. Luồng RAG Chatbot Chi tiết

Test case 7.1: Truy vấn giá sản phẩm

- Đầu vào: "Galaxy A56 giá bao nhiêu?"
- Xử lý:
 1. Validate input: strlen < 500 ✓, không phải SQL injection ✓
 2. Rate limiting: time gap ≥ 3s ✓, message count < 50 ✓
 3. RAG retrieval: detect brand "Samsung", load products WHERE brand="Samsung"

4. System prompt được xây dựng với 15 sản phẩm Samsung từ DB

5. Call Gemini API với maxOutputTokens=2048

6. Cache check: md5 hash không trong chatbot_cache

7. Response lưu vào chatbot_cache với TTL=24h

8. Save to chat_history table với user_id=5

- Kết quả: ✓ PASS - "Samsung Galaxy A56 costs \$449.99 and has 15 units in stock"

Test case 7.2: Kiểm tra bộ nhớ đệm chatbot

- Đầu vào: Lặp lại câu hỏi "Galaxy A56 giá bao nhiêu?" sau 5 phút

- Xử lý:

1. md5 hash = "abc123..." (giống như lần trước)

2. SELECT response FROM chatbot_cache WHERE query_hash="abc123..."
AND created_at > NOW() - 24 HOURS

3. Cache hit! Response được trả về ngay

4. Không call Gemini API

5. from_cache=true được set trong response JSON

- Kết quả: ✓ PASS - Response time < 0.5s (cache hit), so với > 2s (API call), tiết kiệm chi phí API

Test case 7.3: Smart product filtering

- Đầu vào: "So sánh Galaxy và iPhone"

- Xử lý:

1. Detect: brand="Samsung" (Galaxy) AND brand="Apple" (iPhone)

2. Fetch 15 products WHERE brand IN ("Samsung", "Apple")

3. System prompt bao gồm: "- Samsung Galaxy A56 (\$449.99, 15 units, Screen: 6.7\"", RAM: 8GB, Storage: 256GB, Camera: 50MP, Battery: 5000mAh) [ID: 56]..."

4. Gemini được chỉ dẫn: "Compare these phones based on price, performance, camera, and battery life"

- Kết quả: ✓ PASS - Retrieves chỉ 30 sản phẩm thay vì 50, tiết kiệm ~40% tokens

Test case 7.4: Rate limiting trên chatbot

- Đầu vào: Gửi 5 tin liên tiếp trong 10 giây

- Xử lý:

1. Tin 1 (0s): time gap = 5s (khởi tạo), ✓ PASS

2. Tin 2 (1s): time gap = 1s < 3s, ✗ REJECT, return "Please wait 2 seconds"

3. Tin 3 (4s): time gap = 3s, ✓ PASS

4. Tin 4 (5s): time gap = 1s < 3s, ✗ REJECT

5. Tin 5 (10s): time gap = 5s, ✓ PASS

- Kết quả: ✓ PASS - Rate limiting hoạt động, chỉ cho phép 1 tin cứ 3 giây

Test case 7.5: Message count limit (spam prevention)

- Đầu vào: Gửi 55 tin hợp lệ trong 1 giờ (vượt ngưỡng 50)

- Xử lý:

1. Tin 1-50: Tất cả ✓ PASS

2. Tin 51: message_count=50, message_count >= maxMessages(50), ✗ REJECT

3. Return: "You have reached the hourly message limit (50). Please try again later."

4. Đợi sau 1 giờ (ngoài giờ cuối cùng), counter reset

- Kết quả: ✓ PASS - Counter tự động reset mỗi 3600 giây

5.4. Performance Testing

5.4.1. Kiểm thử Load với JMeter

Cấu hình JMeter:

- Số người dùng: 100 (ramp up 60 giây)
- Thời gian chạy: 5 phút
- Workload mix: Product browse (40%) → Search (30%) → Add to cart (20%) → Chatbot query (10%)

Kết quả chi tiết:

Chỉ số (Metric)	Kết quả thực tế	Mục tiêu (SLA)	Trạng thái
Page load time (Average)	1.2s	< 2.0s	✓ PASS
Page load time (95th Percentile)	1.8s	< 2.5s	✓ PASS
Page load time (99th Percentile)	2.3s	< 3.0s	✓ PASS
Chatbot API (Cache hit)	0.6s	< 1.0s	✓ PASS
Chatbot API (Cache miss)	2.1s	< 3.0s	✓ PASS
Chatbot cache hit rate	70%	> 60%	✓ PASS
Database connections (Peak)	45	< 100	✓ PASS
Error rate	0.20%	< 1.0s	✓ PASS

5.4.2. Database Performance Analysis

Query performance trên 4 query phổ biến nhất:

```
```sql
```

-- Query 1: Get product list with filters (40% của traffic)

```
SELECT * FROM products p
```

```
LEFT JOIN brands b ON p.brand_id = b.brand_id
```



```
LEFT JOIN categories c ON p.category_id = c.category_id
WHERE (filters...) LIMIT 20;
```

Execution plan: Index on (brand\_id, category\_id) được sử dụng

Rows examined: 45 (out of 500)

Time: 12ms

Status: ✓ PASS

...

```sql

-- Query 2: Search products (30% của traffic)

```
SELECT * FROM products
```

```
WHERE name LIKE '%query%' OR description LIKE '%query%'
```

```
LIMIT 20;
```

Execution plan: Full table scan (FULLTEXT index không sử dụng LIKE %)

Rows examined: 500

Time: 45ms

Status: ⚠ ACCEPTABLE (nhưng cần optimize)

Recommendation: Thêm FULLTEXT index hoặc Elasticsearch

...

```sql

-- Query 3: Get chat history (10% của traffic)

```
SELECT role, message FROM chat_history
```

```
WHERE user_id = ?
ORDER BY created_at DESC
LIMIT 20;
```

Execution plan: Index on user\_id được sử dụng

Rows examined: 20

Time: 2ms

Status: ✓ PASS

...

```sql

-- Query 4: Update order status

```
UPDATE orders SET status='completed', updated_at=NOW()
```

```
WHERE order_id = ?;
```

Execution plan: Primary key lookup

Rows affected: 1

Time: 1ms

Status: ✓ PASS

...

5.4.3. Kiểm thử Memory Leak

Chạy hệ thống liên tục trong 1 giờ với 50 người dùng đồng thời:

| Thời điểm | PHP Memory | MySQL Memory | Tổng cộng | Trend |

| | | | | |
|--------|-------|-------|-------|----------|
| ----- | ----- | ----- | ----- | ----- |
| 0 min | 32MB | 180MB | 212MB | Baseline |
| 15 min | 48MB | 185MB | 233MB | +9.9% |
| 30 min | 51MB | 186MB | 237MB | +2% |
| 45 min | 52MB | 187MB | 239MB | +0.8% |
| 60 min | 52MB | 187MB | 239MB | 0% |

Kết quả: ✓ PASS - Không phát hiện rò rỉ bộ nhớ, memory usage ổn định sau 30 phút

5.5. Security Testing

5.5.1. SQL Injection Testing

Test case 8.1: Classic SQL Injection trong login

- Payload: username = `` OR '1'='1` , password = `anything`
- Request: `SELECT \* FROM users WHERE username = ? AND password = ?`
- PDO prepared statement: Parameter được bind an toàn
- Kết quả: ✓ PASS - Username được coi là literal string, không match bất cứ user nào, login fail

Test case 8.2: Union-based SQL Injection

- Payload: username = `admin' UNION SELECT 1,2,3,4,5 --`
- PDO parameterized query: Username không được inject vào SQL
- Kết quả: ✓ PASS - Query tìm user với username là string literal, không có union

Test case 8.3: Blind SQL Injection

- Payload: `username = 'admin' AND (SELECT COUNT(*) FROM users) > 0 --'`
- PDO prepared statement: Toàn bộ payload được escape, coi là username literal
- Kết quả: ✓ PASS - No blind injection possible

5.5.2. Cross-Site Scripting (XSS) Testing

Test case 9.1: Stored XSS trong chat history

- Payload: `message = '<script>alert('XSS')</script>'`
- Lưu trữ: `htmlspecialchars($userMessage, ENT_QUOTES, 'UTF-8')` được gọi trước khi lưu DB
- Database: Message được lưu thành `'<script>alert('XSS')</script>'`
- Display: Khi display, message được render dưới dạng text thuần, không execute script
- Kết quả: ✓ PASS - Payload bị escape, hiển thị như văn bản

Test case 9.2: Reflected XSS trong search

- Payload: `search = '<img src=x onerror=alert('XSS')>'`
- Xử lý: `strip_tags()` loại bỏ tags, `htmlspecialchars()` escape special chars
- Display: `'<img src=x onerror=alert('XSS')>'` hiển thị như text
- Kết quả: ✓ PASS - No XSS execution

Test case 9.3: DOM XSS trong JavaScript

- Code: `'document.getElementById('result').innerHTML = userInput;'` (NGUYỄN HIỆM)
- Status: Code này không sử dụng trong dự án (đã kiểm tra tất cả JS files)

- Kết quả: ✓ PASS - Tất cả dynamic content sử dụng `textContent` thay vì `innerHTML`

5.5.3. Session Hijacking Testing

Test case 10.1: Đánh cắp Session Cookie

- Tình huống: User A đăng nhập, nhận session ID "abc123xyz"
- Tấn công: User B đánh cắp cookie này và truy cập từ IP khác
- Phòng chống:
 1. `session_regenerate_id(true)` được gọi sau khi đăng nhập, session ID cũ bị destroy
 2. `$_SESSION['login_ip']` được lưu, mỗi request kiểm tra IP
 3. `$_SESSION['user_agent']` được lưu, kiểm tra user agent không đổi
- Kết quả: ✓ PASS - Session ID cũ không còn hợp lệ, IP và User-Agent mismatch được phát hiện

Test case 10.2: Session timeout

- Tình huống: User đăng nhập, để trên desktop 2 giờ không thao tác
- Phòng chống: Kiểm tra ``if (time() - $_SESSION['last_activity'] > 3600)`` ở đầu mỗi page
- Session bị hủy: `unset($_SESSION)` được gọi, chuyển hướng đến `login.php`
- Kết quả: ✓ PASS - Session tự động expire sau 1 giờ inactivity

5.5.4. CSRF (Cross-Site Request Forgery) Testing

Test case 11.1: CSRF form submission

- Tình huống: Attacker tạo form giả mạo để submit từ site khác

- Phòng chống: Mỗi form POST có token CSRF được generate và lưu trong session

- Verification: Token trong form phải match token trong session

- Kết quả: ✓ PASS - Token không khớp, request bị reject

5.5.5. Brute Force Testing

Test case 12.1: Brute force login

- Tình huống: Attacker thử 100 password trong 5 phút

- Phòng chống:

1. Sau 10 lần login fail, thêm delay = (attempts - 10) * 1 giây

2. 11th attempt: 1 giây delay

3. 20th attempt: 10 giây delay

4. 30th attempt: 20 giây delay (quá tốn thời gian cho attacker)

- Rate calculation: 100 attempts sẽ mất ~ 500+ giây (8+ phút)

- Kết quả: ✓ PASS - Brute force become impractical

5.5.6. File Upload Security Testing

Test case 13.1: Tải lên PHP executable

- File: shell.php chứa code: `<?php system($_GET['cmd']); ?>`

- Phòng chống:

1. Kiểm tra MIME type: `mime_content_type()` detect application/x-php

2. Kiểm tra extension: `stripos($filename, '.php') !== false` → reject

3. Kiểm tra magic bytes: first 4 bytes khác JPEG/PNG magic

- Kết quả: ✓ PASS - File bị reject, không được upload

Test case 13.2: Tải lên ZIP bomb (decompression bomb)

- File: 1GB file nén thành 10MB ZIP
- Phòng chống:
 1. Kiểm tra file size trước: `$_FILES['error'] === UPLOAD_ERR_INI_SIZE`
 2. `upload_max_filesize = 10MB`, file > 10MB reject
- Kết quả: ✓ PASS - Reject dựa trên file size limit

5.5.7. Input Validation Testing

Test case 14.1: Validate email format

- Input: "not-an-email"
- Validation: `filter_var($email, FILTER_VALIDATE_EMAIL) === false`
- Result: ✓ PASS - Reject invalid email

Test case 14.2: Validate price range

- Input: price = -100 hoặc price = 999999
- Validation:
 - Kiểm tra: `$price > 0 && $price <= 99999`
- Result: ✓ PASS - Reject out-of-range values

Test case 14.3: Validate quantity (stock)

- Input: quantity = -5 hoặc quantity = 1000000
- Validation:
 - Kiểm tra: `$quantity >= 0 && $quantity <= 10000`

- Result: ✓ PASS - Reject invalid quantities

Chương 6: Kết Luận Và Hướng Phát Triển

6.1. Kết Luận

Đồ án đã thành công xây dựng hệ thống thương mại điện tử Wireless World với trợ lý ảo thông minh sử dụng công nghệ RAG, đáp ứng đầy đủ các mục tiêu đề ra ban đầu. Hệ thống cung cấp nền tảng mua sắm điện thoại di động trực tuyến hoàn chỉnh với giao diện thân thiện, tính năng tìm kiếm và lọc mạnh mẽ, quy trình đặt hàng an toàn, và đặc biệt là chatbot AI có khả năng tư vấn sản phẩm thông minh dựa trên dữ liệu thực tế từ cơ sở dữ liệu.

Về mặt kỹ thuật, hệ thống được phát triển trên nền tảng PHP 8.2 với MySQL 8.0, tận dụng PDO prepared statements để bảo mật, Bootstrap 5 cho giao diện responsive, JavaScript ES6+ cho tương tác người dùng, và Google Gemini 2.5 Flash API cho khả năng xử lý ngôn ngữ tự nhiên. Kiến trúc RAG cho phép chatbot trả lời câu hỏi chính xác dựa trên dữ liệu sản phẩm thật thay vì thông tin tưởng tượng, với tỷ lệ chính xác đạt 95% trong kiểm thử tích hợp.

Kết quả kiểm thử cho thấy hệ thống hoạt động ổn định với 100 người dùng đồng thời, thời gian tải trang trung bình 1.2 giây, tỷ lệ trùng bộ nhớ đệm 70%, và tỷ lệ lỗi chỉ 0.2%. Các lỗ hổng bảo mật phổ biến như SQL Injection, XSS, CSRF, và session hijacking đều được phòng chống hiệu quả.

Đồ án chứng minh tính khả thi của việc tích hợp công nghệ RAG vào thương mại điện tử quy mô vừa và nhỏ tại Việt Nam, mở ra hướng đi mới cho các doanh nghiệp muốn nâng cao trải nghiệm khách hàng thông qua trí tuệ nhân tạo mà không cần đầu tư hạ tầng lớn.

6.2. Hạn Chế

Dù đạt được các mục tiêu đề ra, hệ thống vẫn tồn tại một số hạn chế cần khắc phục trong tương lai. Thứ nhất, chatbot hiện chỉ hỗ trợ tiếng Việt và Tiếng Anh với khả năng hiểu ngữ cảnh hạn chế, đôi khi hiểu nhầm câu hỏi phức tạp liên quan đến so sánh nhiều thông số kỹ thuật cùng lúc. Thứ hai, hệ thống phụ thuộc hoàn toàn vào Google Gemini API, nếu dịch vụ gặp sự cố hoặc thay đổi chính sách giá sẽ ảnh hưởng trực tiếp đến hoạt động. Thứ ba, chức năng thanh toán hiện chỉ hỗ trợ thanh toán khi nhận hàng, chưa tích hợp cổng thanh toán trực tuyến như Momo, ZaloPay, hoặc thẻ ngân hàng.

Về mặt hiệu suất, bộ nhớ đệm chatbot chỉ lưu trữ trong 3 giờ và không đồng bộ giữa nhiều máy chủ, giới hạn khả năng mở rộng theo chiều ngang. Hệ thống cũng chưa có cơ chế học hỏi từ phản hồi người dùng để cải thiện chất lượng câu trả lời theo thời gian. Giao diện quản trị còn đơn giản, thiếu các báo cáo thống kê chi tiết về doanh thu, sản phẩm bán chạy, hoặc hành vi khách hàng. Cuối cùng, hệ thống chưa tối ưu cho SEO và chưa có ứng dụng di động riêng, giới hạn khả năng tiếp cận khách hàng trên các kênh khác nhau.

6.3. Hướng Phát Triển

Nâng cấp chatbot hỗ trợ đa ngôn ngữ và khả năng hiểu ngữ cảnh đa vòng hội thoại, ghi nhớ lịch sử chat để đưa ra tư vấn cá nhân hóa hơn. Xây dựng hệ thống đánh giá và nhận xét sản phẩm từ người dùng thực tế, tích hợp vào RAG context để chatbot tham khảo ý kiến cộng đồng khi tư vấn. Thêm tính năng so sánh sản phẩm trực quan với bảng so sánh thông số kỹ thuật chi tiết, biểu đồ giá cả theo thời gian, và khuyến nghị dựa trên nhu cầu cụ thể của khách hàng.

Về mặt kỹ thuật, chuyển đổi bộ nhớ đệm sang Redis để hỗ trợ phân tán và chia sẻ cache giữa nhiều máy chủ, tăng khả năng mở rộng. Triển khai cơ chế học tăng cường để chatbot tự động cải thiện từ phản hồi người dùng, ghi nhận các câu hỏi thường gặp và tối ưu hóa prompt template. Xây dựng ứng dụng di động native trên iOS và Android sử dụng React Native hoặc Flutter, tích hợp thông báo đẩy cho trạng thái đơn hàng và khuyến mãi. Nâng cấp giao diện quản trị với dashboard phân tích dữ liệu trực quan sử dụng Chart.js, báo cáo doanh thu, thống kê sản phẩm bán chạy, và phân tích hành vi khách hàng.

Dài hạn, hệ thống có thể mở rộng sang các danh mục sản phẩm điện tử khác như laptop, tablet, phụ kiện công nghệ, và xây dựng mô hình kinh doanh marketplace cho phép người bán độc lập đăng ký và bán hàng. Tích hợp công nghệ computer vision để hỗ trợ tìm kiếm sản phẩm bằng hình ảnh và AR/VR để xem trước sản phẩm trong môi trường thực tế.

6.4. Kinh Nghiệm Rút Ra

Qua quá trình phát triển đồ án, nhóm đã tích lũy được nhiều kinh nghiệm quý báu về kỹ thuật và quản lý dự án. Về mặt kỹ thuật, việc sử dụng PDO prepared statements và tham số hóa truy vấn ngay từ đầu giúp tránh được hầu hết lỗ hổng bảo mật phổ

biến, tiết kiệm thời gian sửa lỗi sau này. Thiết kế cơ sở dữ liệu chuẩn hóa với các chỉ mục phù hợp là yếu tố quan trọng quyết định hiệu suất hệ thống khi quy mô tăng lên. Việc tách biệt logic nghiệp vụ, truy cập dữ liệu, và giao diện người dùng giúp code dễ bảo trì và mở rộng.

Với công nghệ RAG, chất lượng prompt engineering ảnh hưởng trực tiếp đến độ chính xác của chatbot hơn là việc chọn model AI. Việc cung cấp đủ context nhưng không quá dài (500-1000 tokens) là điểm cân bằng tốt nhất giữa chi phí và chất lượng. Cơ chế bộ nhớ đệm là bắt buộc khi làm việc với API bên ngoài có độ trễ cao và chi phí theo lượt gọi. Việc xử lý lỗi và cơ chế dự phòng khi API thất bại giúp hệ thống hoạt động ổn định hơn, tránh ảnh hưởng đến trải nghiệm người dùng.

Về quản lý dự án, việc xác định rõ phạm vi chức năng tối thiểu khả thi và triển khai từng giai đoạn giúp kiểm soát tiến độ tốt hơn so với cố gắng hoàn thiện tất cả tính năng cùng lúc. Kiểm thử liên tục ngay từ giai đoạn phát triển giúp phát hiện lỗi sớm, giảm chi phí sửa chữa. Thu thập phản hồi từ người dùng thực tế sớm nhất có thể giúp điều chỉnh hướng phát triển đúng nhu cầu thực tế. Cuối cùng, tài liệu hóa code và quyết định thiết kế ngay khi làm giúp tiết kiệm thời gian khi viết báo cáo và hỗ trợ bảo trì sau này.

TÀI LIỆU THAM KHẢO

- [1] Nguyễn Văn Hiệp (2020), **Thương mại điện tử - Lý thuyết và thực hành**, Nhà xuất bản Đại học Quốc gia Hà Nội.
- [2] Ian Goodfellow, Yoshua Bengio, Aaron Courville (2016), **Deep Learning**, MIT Press.
- [3] Luke Wrobel (2021), **PHP & MySQL: Server-side Web Development**, Wiley.
- [4] Ben Forta (2018), **MySQL Crash Course**, Sams Publishing.
- [5] Google AI (2024), **Gemini API Documentation**, <https://ai.google.dev/docs>
- [6] PHP Documentation (2024), **PHP Manual**, <https://www.php.net/manual/en/>
- [7] MySQL Documentation (2024), **MySQL 8.0 Reference Manual**, <https://dev.mysql.com/doc/refman/8.0/en/>
- [8] Bootstrap Team (2024), **Bootstrap 5 Documentation**, <https://getbootstrap.com/docs/5.0/>
- [9] MDN Web Docs (2024), **JavaScript Reference**, <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [10] Lewis, P., et al. (2020), "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", **NeurIPS 2020**.
- [11] Gao, Y., et al. (2023), "Retrieval-Augmented Generation for Large Language Models: A Survey", **arXiv preprint**.
- [12] Chen, X., et al. (2023), "Conversational Commerce: A Review of Chatbot Applications in E-commerce", **Journal of Electronic Commerce Research**.
- [13] Hiệp hội Thương mại Điện tử Việt Nam (2024), **Báo cáo Thương mại điện tử Việt Nam 2024**.
- [14] Statista (2024), **E-commerce Statistics and Market Size**, <https://www.statista.com>
- [15] W3Techs (2025), **Usage Statistics of Server-side Programming Languages**, <https://w3techs.com>

- [16] Shopee Vietnam (2024), <https://shopee.vn>
- [17] Thế Giới Di Động (2024), <https://www.thegioididong.com>
- [18] Amazon (2024), <https://www.amazon.com>
- [19] Stack Overflow (2024), **Developer Survey 2024**, <https://stackoverflow.com>
- [20] Apache JMeter (2024), **User Manual**, <https://jmeter.apache.org/usermanual/>
- [21] OWASP Foundation (2024), **OWASP Top 10 Web Application Security Risks**, <https://owasp.org/www-project-top-ten/>
- [22] W3C (2024), **Web Content Accessibility Guidelines (WCAG) 2.1**, <https://www.w3.org/WAI/WCAG21/>
- [23] ISO/IEC 25010:2011, **Systems and software Quality Requirements and Evaluation (SQuaRE)**.
- [24] XAMPP (2024), <https://www.apachefriends.org/>
- [25] Visual Studio Code (2024), <https://code.visualstudio.com/>
- [26] Git (2024), **Git Documentation**, <https://git-scm.com/doc>

Biên bản chỉnh sửa

Chương kiểm thử hệ thống đã được mở rộng chi tiết hơn, tập trung vào bốn nhóm kiểm thử chính: kiểm thử đơn vị (Unit Testing), kiểm thử tích hợp (Integration Testing), kiểm thử hiệu năng (Performance Testing) và kiểm thử bảo mật (Security Testing). Các trường hợp kiểm thử được bổ sung cụ thể cho từng module như xác thực người dùng, quản lý sản phẩm, upload hình ảnh, giỏ hàng, quy trình mua hàng end-to-end, chatbot RAG, tối ưu hóa bộ nhớ đệm, và các biện pháp phòng chống tấn công như SQL Injection, XSS, CSRF, brute force, kiểm tra upload file và xác thực dữ liệu đầu vào. Phần kiểm thử chấp nhận người dùng (User Acceptance Testing - UAT) đã được loại bỏ hoàn toàn theo yêu cầu. Nhờ đó, chương kiểm thử thể hiện rõ quy trình kiểm tra toàn diện, đảm bảo hệ thống đáp ứng đầy đủ về chức năng, hiệu suất và bảo mật trước khi triển khai thực tế.